

**VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY
OF TECHNOLOGY**



MINI-PROJECT

Embedded System Programming

PROJECT

Design and Implementation of an STM32-based Thermostat Controller

CLASS CC01 --- HK252

Instructor: MSc. Bùi Quốc Bảo

No.	Student's name	Student's ID	Task Description	Status
1	Nguyễn Tiến Anh	2351005	Firmware development & simulation	
2	Phạm Thành Vinh	2351075	Report writing	

Ho Chi Minh City – May 2026

Table of Contents

1. Introduction	8
1.1 Project Background	8
1.2 Project Objectives	8
1.3 Report Organization	9
2. Requirement Analysis	10
2.1 Problem Statement	10
2.2 Functional Requirements	10
2.2.1 Temperature Measurement	11
2.2.2 Temperature Setpoint Adjustment	12
2.2.3 ON/OFF Fan Control	12
2.2.4 LCD Display	13
2.2.5 Button-based User Interface	13
2.3 Non-functional Requirements	14
2.3.1 Timing Requirement	14
2.3.2 Temperature Resolution Requirement	15
2.3.3 Code Modularity Requirement	15
2.3.4 Simulation Requirement	16
2.4 Requirement-to-Module Mapping	16
3. System Overview	17
3.1 Overall System Description	17
3.2 System Block Diagram	18
3.3 Hardware Components	19
3.4 Pin Assignment	19
4. Software Architecture	20

4.1 Software Design Approach	20
4.2 Layered Software Architecture	21
4.2.1 Application Layer	21
4.2.2 Board Support Package Layer	21
4.2.3 Driver / Component Layer	22
4.2.4 Hardware Layer	22
4.3 Project Folder Structure	22
4.4 Advantages of the Modular Structure	24
5. Detailed Software Design	25
5.1 Main Program Flow	25
5.2 System Initialization Flow	26
5.3 Super Loop Design.....	28
5.4 Timing Design.....	29
5.4.1 SysTick-based Time Base.....	30
5.4.2 500 ms Temperature Reading Period	30
5.5 Button Handling Design.....	31
5.5.1 Button Functions.....	31
5.5.2 Button Debouncing	32
5.5.3 Setpoint Increase / Decrease Logic.....	32
5.6 Temperature Reading Design.....	33
5.6.1 DS18B20 Communication Overview	33
5.6.2 Temperature Conversion	34
5.6.3 Temperature Resolution Handling.....	34
5.6.4 Non-blocking Temperature Acquisition	35
5.7 Fan Control Design	36
5.7.1 ON/OFF Control Algorithm	36

5.7.2 Fan ON Condition.....	36
5.7.3 Fan OFF Condition	36
5.8 LCD Display Design	37
5.8.1 Displayed Information	37
5.8.2 LCD Update Strategy	37
5.8.3 Fan Status Display	37
5.9 Thermostat State Machine Design	38
5.9.1 State Definition	38
5.9.2 State Transition Diagram	39
5.9.3 State Transition Table.....	39
7. Proteus Simulation	40
7.1 Simulation Circuit Overview	40
7.2 Proteus Schematic	41
7.3 Simulation Components	41
7.4 Simulation Setup	42
7.5 Expected Simulation Behavior.....	43
8. Testing and Results.....	43
8.1 Testing Methodology	43
8.2 Functional Test Cases	45
8.2.1 System Startup Test	45
8.2.2 Temperature Reading Test.....	46
8.2.3 Setpoint Adjustment Test	47
8.2.4 Fan ON/OFF Control Test	48
8.2.5 LCD Display Test	50
8.2.6 500 ms Update Timing Test	51
8.3 Test Result Table.....	51

8.4 Simulation Screenshots	53
8.4.1 Initial System State	53
8.4.2 Temperature Below Setpoint	54
8.4.3 Temperature Above Setpoint	55
8.4.4 Fan ON State	56
8.4.5 Fan OFF State	56
8.4.6 Setpoint Adjustment using Buttons	57
8.5 Requirement Verification Matrix	57
9. Discussion	59
9.1 Achieved Results	59
9.2 Limitations of the Current System	59
9.3 Possible Improvements	60
9.3.1 Hysteresis Control	60
9.3.2 UART Debugging	60
9.3.3 Sensor Error Detection	61
9.3.4 Saving Setpoint to Flash Memory	61
9.3.5 RTOS-based Extension	62
10. Conclusion	63
10.1 Project Summary	63
10.2 Learning Outcomes	63
10.3 Final Remarks	63
References.....	64
Appendix.....	64
Appendix A. Full Project Folder Structure	64
Appendix B. Full Pin Assignment Table	66

Appendix C. Important Source Code	67
Appendix D. Proteus Schematic.....	76
Appendix E. Additional Simulation Results	77

LIST OF FIGURES

Figure Number	Description
Figure 3.1	System block diagram of the STM32-based thermostat controller
Figure 5.1	Main program flow of the thermostat software
Figure 5.2	System initialization flow of the thermostat application
Figure 5.3	Super-loop execution flow of the thermostat firmware
Figure 5.4	DS18B20 temperature sampling timeline
Figure 5.5	Non-blocking DS18B20 driver state machine
Figure 5.6	Thermostat application state transition diagram
Figure 7.1	Complete Proteus schematic of the thermostat system
Figure 8.1	Initial system state after startup
Figure 8.2	Initial system state in Proteus
Figure 8.3	Temperature below setpoint
Figure 8.4	Temperature above setpoint
Figure 8.5	Fan ON state

Figure 8.6	Fan OFF state
Figure 8.7	Setpoint adjustment using buttons
Figure D.1	Full Proteus schematic of the STM32-based thermostat controller
Figure E.1	Simulation result in normal mode with fan ON
Figure E.2	Setpoint setting mode with 25°C target temperature
Figure E.3	Simulation result in normal mode with fan OFF
Figure E.4	Setpoint setting mode with 29°C target temperature

1. Introduction

1.1 Project Background

Temperature control is a common function in many embedded systems, especially in devices such as air conditioners, refrigerators, heaters, and environmental control systems. In these applications, a microcontroller is usually responsible for reading temperature data from a sensor, processing the measured value, and controlling an output device according to the desired operating condition.

A thermostat is a simple but practical example of this type of embedded control system. It continuously monitors the surrounding temperature and activates or deactivates a cooling or heating device based on a preset temperature value. Although the control principle is simple, a thermostat still requires the integration of several important embedded system functions, including sensor reading, GPIO control, user input handling, display control, timing management, and system-level decision making.

In this mini-project, an STM32-based thermostat controller is developed as a practical application for the Embedded System Programming course. The system uses a DS18B20 digital temperature sensor to measure room temperature, push buttons to adjust the temperature setpoint, an LCD to display system information, and a fan output to represent the cooling actuator. The controller is implemented in C using STM32CubeIDE and verified through Proteus simulation.

From the learning perspective, this project provides an opportunity to apply core embedded programming concepts in a complete system. Instead of only testing individual peripherals, the project focuses on integrating multiple modules into one working application. It also emphasizes structured firmware design, where the application logic, board support modules, and low-level driver components are separated to improve readability and maintainability.

1.2 Project Objectives

The main objective of this project is to design, implement, and simulate an STM32-based thermostat controller that can monitor room temperature and control a cooling fan automatically.

The specific objectives are as follows:

First, the system should acquire temperature data from a DS18B20 sensor and process the measured value for thermostat control. The temperature value is used as the main input for deciding whether cooling is required.

Second, the system should allow the user to adjust the desired temperature setpoint through push buttons. This provides a simple user interface without requiring a computer terminal or external communication device.

Third, the system should control the fan using an ON/OFF control algorithm. When the measured temperature is higher than the selected setpoint, the fan is turned on. When the measured temperature is equal to or lower than the setpoint, the fan is turned off.

Fourth, the system should display important operating information on an LCD, including the current temperature, selected setpoint, and fan status. This allows the user to observe the thermostat behavior during simulation.

Fifth, the firmware should be organized into clear software modules. The application layer handles the thermostat behavior, while the BSP and driver layers handle hardware-related operations such as button reading, sensor communication, LCD control, fan control, and timing services.

Finally, the completed system should be verified in Proteus simulation to confirm that the temperature reading, setpoint adjustment, LCD display, and fan control functions work correctly as an integrated embedded application.

1.3 Report Organization

This report is organized into ten main chapters.

Chapter 1 introduces the project background, objectives, and overall report structure.

Chapter 2 presents the requirement analysis of the thermostat system. It describes the problem statement, functional requirements, non-functional requirements, and the mapping between requirements and software modules.

Chapter 3 provides the system overview. It introduces the overall system operation, hardware components, block diagram, pin assignment, and development tools used in the project.

Chapter 4 describes the software architecture. This chapter explains the layered software design, including the Application layer, Board Support Package layer, Driver or Component layer, and Hardware layer. It also presents the project folder structure and the responsibility of each software module.

Chapter 5 presents the detailed software design. It explains the main program flow, initialization flow, super loop design, timing design, button handling, temperature reading, fan control, LCD display strategy, and thermostat state machine.

Chapter 6 describes the implementation of the system. It discusses the STM32CubeIDE project configuration, GPIO setup, application layer implementation, BSP modules, driver components, main control algorithm, and important code snippets.

Chapter 7 presents the Proteus simulation. It describes the simulation circuit, simulation components, setup procedure, and expected system behavior.

Chapter 8 presents the testing and results. It includes functional test cases, test result tables, simulation screenshots, and a requirement verification matrix.

Chapter 9 discusses the achieved results, limitations of the current system, and possible improvements such as hysteresis control, UART debugging, sensor error detection, Flash memory storage, and RTOS-based extension.

Chapter 10 concludes the report by summarizing the project outcomes and the main learning results from designing and implementing the STM32-based thermostat controller.

2. Requirement Analysis

2.1 Problem Statement

The main problem is to integrate several hardware-related functions into a stable embedded control application. The firmware must periodically acquire temperature data, process user input, update the LCD, and control the fan without blocking the main loop. Therefore, the system requires clear task scheduling, reliable button handling, and modular software organization.

2.2 Functional Requirements

Functional requirements describe what the system must do during operation. For this thermostat project, the main functional requirements are related to temperature measurement, setpoint adjustment, fan control, LCD display, and button-based user interaction.

The functional requirements of the system are summarized in Table 2.1.

Table 2.1. Functional Requirements

ID	Functional Requirement	Description
FR1	Temperature Measurement	The system shall read the current room temperature from the DS18B20 temperature sensor.

FR2	Temperature Setpoint Adjustment	The system shall allow the user to adjust the desired temperature setpoint.
FR3	ON/OFF Fan Control	The system shall control the fan based on the comparison between the current temperature and the setpoint.
FR4	LCD Display	The system shall display the current temperature and fan status on the LCD.
FR5	Button-based User Interface	The system shall use push buttons as the input interface for changing the setpoint.

2.2.1 Temperature Measurement

The thermostat must measure the current room temperature using a DS18B20 digital temperature sensor. The measured temperature is the main input of the control algorithm. The system uses this value to decide whether the fan should be turned ON or OFF.

The required measurement range is from 0°C to 60°C. This range is suitable for a room-temperature monitoring application and covers the expected operating conditions of the thermostat. The temperature value used by the application is represented with 1°C resolution, meaning that the displayed and processed value is treated as an integer temperature value in degrees Celsius.

The system must update the measured temperature periodically. Based on the assignment requirement, the reading period is 500 ms. Therefore, the software must include a timing mechanism to make sure the temperature reading task is executed regularly rather than randomly in the main loop.

2.2.2 Temperature Setpoint Adjustment

The system must provide a way for the user to set the desired temperature threshold. This threshold is called the temperature setpoint. The setpoint represents the temperature that the thermostat tries to maintain through fan control.

In this project, the setpoint is adjusted using push buttons. A typical user interface includes one button to increase the setpoint, one button to decrease the setpoint, and one button to confirm or switch the setting mode if needed. The exact behavior depends on the final implementation, but the main requirement is that the user must be able to change the target temperature during operation.

The setpoint adjustment function is important because it allows the thermostat to operate according to the user's desired comfort level. Without this feature, the system would only work with a fixed temperature threshold and would not satisfy the assignment requirement for a user interface.

2.2.3 ON/OFF Fan Control

The fan is the main output actuator of the thermostat system. It represents the cooling action of the air-conditioning system. The control algorithm used in this project is an ON/OFF control algorithm, which is simple and suitable for a basic embedded thermostat.

The basic control logic is as follows:

If current temperature > set temperature:
 Turn fan ON
Else:
 Turn fan OFF

When the current temperature is higher than the setpoint, the system turns ON the fan to simulate cooling. When the current temperature becomes lower than or equal to the setpoint, the system turns OFF the fan. This behavior allows the thermostat to automatically respond to temperature changes.

The assignment specifically requires the system to control the cooling fan using an ON/OFF algorithm and display the ON/OFF status of the fan during operation.

Table 2.2. Fan Control Behavior

Current Temperature	Set Temperature	Expected Fan State
32°C	28°C	ON
28°C	28°C	OFF
25°C	28°C	OFF

2.2.4 LCD Display

The LCD is used as the main output interface for the user. During operation, the LCD should show the current temperature and the fan status. Depending on the final layout of the display, it may also show the set temperature.

The LCD display function helps the user observe the system behavior directly. For example, when the temperature changes, the current temperature value on the LCD should be updated. When the fan state changes from OFF to ON or from ON to OFF, the fan status displayed on the LCD should also change accordingly.

A possible LCD display format is shown below:

Temp: 28 C Set: 26 C Fan: ON

This display format provides enough information for the user to understand the current state of the thermostat, including the measured temperature, the selected setpoint, and the actuator status.

2.2.5 Button-based User Interface

The button-based user interface allows the user to interact with the thermostat without using a computer or external terminal. The buttons are mainly used to change the temperature setpoint.

A typical button assignment is shown in Table 2.3.

Table 2.3. Button Functions

Button	Function
UP	Increase the temperature setpoint
DOWN	Decrease the temperature setpoint
SET	Confirm or enter setting mode

Because mechanical buttons may generate unstable signals when pressed or released, the software should include a debounce mechanism. Debouncing helps prevent one physical button press from being detected as multiple presses. This makes the setpoint adjustment more stable and reliable.

2.3 Non-functional Requirements

Non-functional requirements describe the qualities and constraints of the system. These requirements do not directly describe a specific function, but they affect the reliability, readability, maintainability, and testability of the software.

The non-functional requirements of this project are summarized in Table 2.4.

Table 2.4. Non-functional Requirements

ID	Non-functional Requirement	Description
NFR1	Timing Requirement	The system shall read temperature every 500 ms.
NFR2	Temperature Resolution Requirement	The system shall process and display temperature with 1°C resolution.
NFR3	Code Modularity Requirement	The firmware shall be divided into clear modules such as App, BSP, and Driver components.
NFR4	Simulation Requirement	The system shall be testable in Proteus simulation.

2.3.1 Timing Requirement

The system must read the temperature every 500 ms. This timing requirement ensures that the thermostat updates the measured temperature frequently enough to respond to temperature changes. If the reading interval is too long, the system may respond slowly. If the reading interval is too short, the software may waste processing time or cause unnecessary updates.

In the firmware design, the timing requirement can be implemented using a SysTick-based time base or a software timer. Instead of blocking the entire program with long delays, the application should check whether 500 ms has elapsed before starting a new temperature reading. This approach is more suitable for a super-loop embedded application because other tasks, such as button handling and LCD updating, can still be executed repeatedly.

2.3.2 Temperature Resolution Requirement

The assignment requires a temperature resolution of 1°C. This means the system does not need to display or process fractional temperature values such as 28.5°C or 28.75°C. Instead, the temperature can be rounded or converted to an integer value before being displayed and used by the control algorithm.

Using 1°C resolution simplifies the display and control logic. It also makes the system easier for the user to read on a small LCD screen. Since the thermostat is designed as a simple room temperature controller, 1°C resolution is acceptable for the scope of this assignment.

2.3.3 Code Modularity Requirement

The firmware should be organized into separate modules. This requirement is important because the project includes multiple hardware components and software functions. If all code is placed directly in `main.c`, the program will become difficult to read, debug, and modify.

The course material emphasizes that a large C program is typically divided into multiple modules. Each module receives input data, processes it, and produces output results. A module usually includes a header file `.h` for function declarations and shared definitions, and a source file `.c` for function implementations.

For this project, the modular structure includes:

Module	Main Responsibility
App	Main thermostat logic
BSP_Button	Button reading and debounce
BSP_DS18B20	Temperature sensor interface
BSP_Fan	Fan control
BSP_LCD	LCD control
delay	Delay service
dio	Digital input/output abstraction
systick	System tick and timing base

The use of modules improves maintainability because each module can be developed and tested separately. It also makes the code easier to adapt if the hardware pins or board are changed.

2.3.4 Simulation Requirement

The thermostat prototype must be tested using Proteus simulation. The simulation should include the STM32 board, DS18B20 sensor, LCD, push buttons, and fan output. The purpose of simulation is to verify the system behavior before applying it to real hardware.

Through simulation, the following behaviors should be checked:

Simulation Item	Expected Behavior
Temperature sensor	Temperature value can be changed and read by the MCU
Buttons	Setpoint changes when buttons are pressed
Fan output	Fan turns ON/OFF according to the control algorithm
LCD	Current temperature and fan status are displayed
Timing	Temperature is updated every 500 ms

Proteus simulation is useful because it allows the group to observe the interaction between software and hardware components. It also helps detect wiring, GPIO, and logic problems before hardware testing.

2.4 Requirement-to-Module Mapping

To make sure every requirement is covered by the implementation, each requirement is mapped to one or more software modules. This mapping also helps explain the role of each module in the project.

Table 2.5. Requirement-to-Module Mapping

Requirement ID	Requirement	Related Module
FR1	Read current temperature	BSP_DS18B20, delay, dio
FR2	Adjust temperature setpoint	BSP_Button, App
FR3	Control fan using ON/OFF algorithm	App, BSP_Fan, dio

FR4	Display current temperature and fan status	BSP_LCD, App
FR5	Button-based user interface	BSP_Button, App
NFR1	500 ms temperature reading period	Systick, App
NFR2	1°C temperature resolution	BSP_DS18B20, App
NFR3	Modular code organization	App, BSP, Drivers/Components
NFR4	Proteus simulation	Full integrated system

The mapping shows that the thermostat is implemented as a combination of application logic and hardware-related modules. The App module coordinates the overall behavior, while the BSP modules hide the details of each hardware component. This follows the layered design concept introduced in the course, where the application controls peripherals by calling Board Support Package functions, and the BSP can call lower-level driver APIs when needed.

3. System Overview

This chapter presents the overall structure of the embedded thermostat system. The system is built around an STM32F401CEUx microcontroller and several external peripherals, including a DS18B20 digital temperature sensor, a 16x2 HD44780-compatible LCD, push buttons, and a fan or cooling actuator output. The chapter describes the system operation, hardware structure, main components, and GPIO pin assignment used in the implemented system.

3.1 Overall System Description

The developed system is an embedded thermostat designed to monitor room temperature and control a cooling output according to a user-defined setpoint. The system is built around an STM32F401CEUx microcontroller, which coordinates temperature measurement, user input processing, display update, and actuator control.

Temperature is measured by a DS18B20 digital temperature sensor. The user interacts with the system through push buttons, while a 16x2 LCD displays the current temperature, the selected setpoint, and the operating status of the thermostat. A digital output from the microcontroller is used to control a fan or cooling actuator through an external driver or relay circuit.

The thermostat performs basic ON/OFF temperature control. When the measured temperature is higher than the selected setpoint, the cooling output is activated. When the measured temperature is equal to or lower than the setpoint, the cooling output is deactivated. This control method is simple and suitable for the required thermostat application.

The system is implemented using a modular software structure so that the application logic, board-specific connections, and reusable drivers are clearly separated. This organization improves code readability and makes the project easier to maintain and extend.

3.2 System Block Diagram

The thermostat consists of five main functional blocks: temperature sensing, user input, processing, display, and actuation. The STM32F401CEUx microcontroller is the central processing unit and interfaces with all external peripherals through GPIO signals.

The DS18B20 sensor sends digital temperature data to the STM32 through a 1-Wire interface. Push buttons provide user input for operating the thermostat and adjusting the setpoint. The LCD receives display data from the STM32 using a 4-bit parallel interface. The fan or cooling actuator is controlled by a GPIO output signal from the microcontroller.

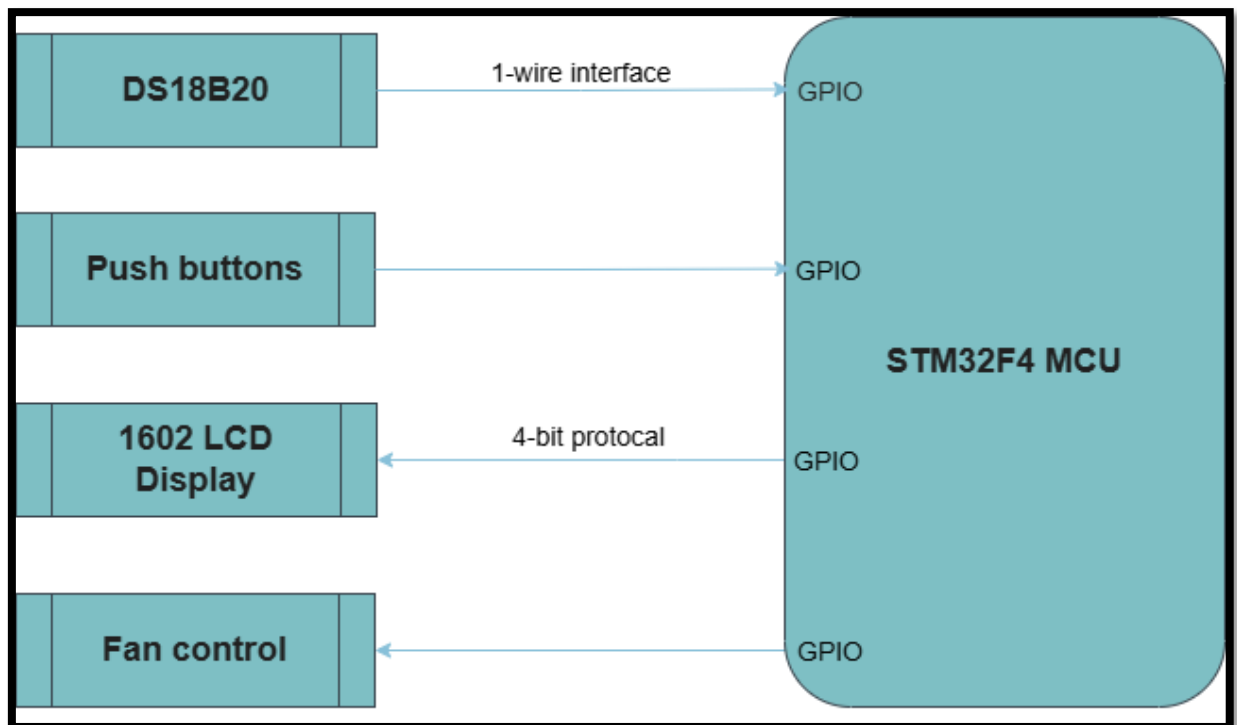


Figure 3.1: System block diagram of the STM32-based thermostat controller

3.3 Hardware Components

The main hardware components used in the thermostat system are summarized in the following table

Component	Function in the system
STM32F401CEUx microcontroller	Main controller that performs temperature processing, user input handling, display control, and actuator control
DS18B20 temperature sensor	Measures ambient temperature and sends digital data through the 1-Wire interface
16x2 HD44780-compatible LCD	Displays current temperature, setpoint, and thermostat status
Push buttons	Provide user input for mode selection and setpoint adjustment
Fan / cooling actuator output	Represents the controlled cooling device of the thermostat

3.4 Pin Assignment

The physical connection between the STM32F401CEUx microcontroller and the external devices is defined in the BSP files. The BSP layer contains the board-specific pin mapping and creates the driver objects used by the application.

Peripheral	Signal	STM32 Port/Pin	Direction	Description
LCD	RS	PA2	Output	Register select signal for LCD command/data selection
LCD	EN	PA3	Output	Enable signal used to latch LCD data
LCD	D4	PA4	Output	LCD 4-bit data line
LCD	D5	PA5	Output	LCD 4-bit data line
LCD	D6	PA6	Output	LCD 4-bit data line

LCD	D7	PA7	Output	LCD 4-bit data line
DS18B20	DQ	PB10	Input/Output	1-Wire data line with pull-up resistor
Push Button	UP	PB0	Input	Increases the set temperature in setting mode
Push Button	DOWN	PB1	Input	Decreases the set temperature in setting mode
Push Button	SET	PB2	Input	Enters setting mode and confirms the edited setpoint
Push Button	POWER	PB3	Input	Switches between operating modes
Fan	FAN_CONTROL	PB5	Output	Controls the fan or cooling actuator

The LCD signals are grouped on GPIOA from PA2 to PA7, which simplifies the physical wiring. The DS18B20 sensor uses PB10 as a bidirectional data line. The user input buttons are connected to GPIOB pins PB0 to PB3, while PB5 is used as the fan control output.

4. Software Architecture

This chapter describes the software architecture of the embedded thermostat system. The software is implemented in C using a modular bare-metal structure. The design separates the application logic, board-specific wiring, device drivers, and low-level hardware access functions. This structure improves readability, maintainability, and reusability of the project.

4.1 Software Design Approach

The thermostat software is developed using a modular bare-metal design. Instead of placing all functions in one source file, the program is divided into separate software layers according to responsibility. High-level application behavior is separated from board-specific wiring and low-level device control.

The software runs without an operating system. The application is executed using a super-loop structure, while timing services are provided by a 1 ms SysTick-based time base. The detailed execution flow, timing strategy, button processing, temperature acquisition, and state-machine behavior are presented later in Chapter 5.

The design follows two main principles:

Separation of responsibility: each module performs a clearly defined task.

Hardware abstraction: reusable drivers are kept independent from the exact physical pin assignment of the board.

This approach makes the software easier to understand, test, modify, and reuse in future projects.

4.2 Layered Software Architecture

The thermostat software is organized into four main layers:

Application layer : Thermostat behavior and system control

Board Support Package layer : Board-specific pin mapping and object binding

Driver / component layer : Reusable device drivers and support components

Hardware layer : STM32F401CEUx and external peripherals

This layered organization separates system behavior from hardware-dependent implementation details.

4.2.1 Application Layer

The application layer contains the high-level thermostat behavior and is implemented in `app_main.c`. It coordinates the operating modes of the system, temperature supervision, user interaction, display update, and fan control.

This layer uses the services provided by lower layers but does not directly define the physical hardware connections. As a result, the main thermostat logic remains separated from the board wiring details.

4.2.2 Board Support Package Layer

The Board Support Package layer defines how the reusable drivers are connected to the actual STM32 pins used in this project. It contains the following modules:

<code>bsp_button</code> <code>bsp_ds18b20</code> <code>bsp_fan</code> <code>bsp_lcd</code>

Each BSP module creates and configures the objects used by the application, such as button objects, the DS18B20 sensor object, the fan object, and the LCD object. The BSP layer contains board-specific information such as GPIO ports, pin numbers, active levels, and initial configuration values.

This layer is kept intentionally simple. If the hardware wiring is changed, only the BSP files need to be updated, while the application and driver code can remain unchanged.

4.2.3 Driver / Component Layer

The driver layer contains reusable modules that provide peripheral control and low-level services. In the project structure, this layer is divided into two groups:

- Components
- Devices

The Components group contains general support modules:

delay dio systick

These modules provide timing services and basic digital I/O support.

The Devices group contains drivers for the external devices used by the thermostat:

button ds18b20 fan lcd

These drivers implement the behavior of each peripheral while remaining independent from the physical board wiring. For example, the LCD driver provides display functions, but the actual LCD pin connections are defined separately in the BSP layer.

The detailed internal design of these modules is presented later in Chapter 5 and Chapter 6.

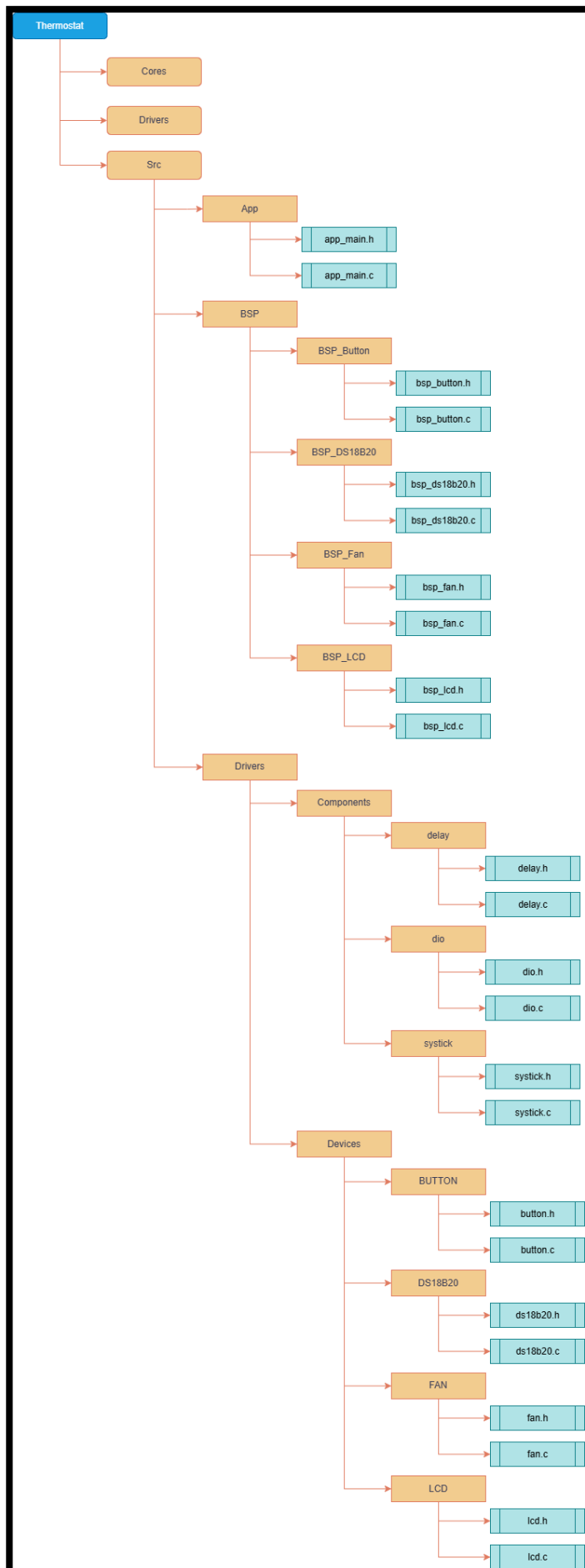
4.2.4 Hardware Layer

The hardware layer consists of the STM32F401CEUx microcontroller and the external devices connected to it, including the DS18B20 sensor, LCD, push buttons, and cooling actuator output.

The upper software layers interact with the hardware through STM32 registers and GPIO signals. By placing the hardware-dependent configuration below the application layer, the software structure becomes easier to maintain and less dependent on one specific board layout.

4.3 Project Folder Structure

The custom software of the thermostat project is organized under the Src directory. The folders are arranged according to the layered software architecture:



```

Thermostat
├── Binaries
├── Includes
├── Core
├── Drivers
├── Src
│   ├── App
│   │   ├── app_main.c
│   │   └── app_main.h
│   ├── BSP
│   │   ├── BSP_Button
│   │   │   ├── bsp_button.c
│   │   │   └── bsp_button.h
│   │   ├── BSP_DS18B20
│   │   │   ├── bsp_ds18b20.c
│   │   │   └── bsp_ds18b20.h
│   │   ├── BSP_Fan
│   │   │   ├── bsp_fan.c
│   │   │   └── bsp_fan.h
│   │   └── BSP_LCD
│   │       ├── bsp_lcd.c
│   │       └── bsp_lcd.h
│   └── Drivers
│       ├── Components
│       │   ├── delay
│       │   │   ├── delay.c
│       │   │   └── delay.h
│       │   ├── dio
│       │   │   ├── dio.c
│       │   │   └── dio.h
│       │   └── systick
│       │       ├── systick.c
│       │       └── systick.h
│       └── Devices
│           ├── BUTTON
│           │   ├── button.c
│           │   └── button.h
│           ├── DS18B20
│           │   ├── ds18b20.c
│           │   └── ds18b20.h
│           ├── FAN
│           │   ├── fan.c
│           │   └── fan.h
│           └── LCD
│               ├── lcd.c
│               └── lcd.h
  
```

The custom source code is divided into three main directories:

Folder	Purpose
App	Contains the high-level thermostat application logic
BSP	Contains board-specific pin mapping modules
Drivers	Contains reusable component modules and device drivers

Inside Drivers, the software is further divided into:

Subfolder	Contents
Components	delay, dio, and systick modules
Devices	button, ds18b20, fan, and lcd drivers

This structure directly reflects the logical software organization of the project. Application behavior is placed separately from board wiring, and reusable drivers are grouped according to whether they provide general services or device-specific functions.

4.4 Advantages of the Modular Structure

The modular software structure provides several practical advantages for the thermostat project.

First, it improves readability because each file has a clear responsibility. The application logic, hardware wiring, and device-control functions are not mixed together in the same module.

Second, it improves maintainability. For example, if the DS18B20 pin or LCD connection changes, only the corresponding BSP file needs to be modified. The reusable driver and application logic do not need to be rewritten.

Third, it improves reusability. Drivers such as the LCD, button, fan, and DS18B20 modules can be reused in another embedded project by providing a different BSP configuration.

Finally, the modular structure makes debugging easier. Problems can be isolated more effectively because each layer has a defined function. Hardware connection issues can be checked in the BSP layer, device behavior can be checked in the driver layer, and operating logic can be checked in the application layer.

5. Detailed Software Design

This chapter presents the internal software design of the thermostat system. While the previous chapter described the overall software architecture, this chapter focuses on how the main functions are implemented, including the program flow, initialization sequence, periodic scheduling, button handling, DS18B20 temperature acquisition, fan control, LCD display update, and thermostat state machine.

5.1 Main Program Flow

The thermostat software follows a bare-metal super-loop design. After the microcontroller and low-level services are initialized, the application initialization function `app_main_init()` is executed once. The program then repeatedly calls `app_main_run()` inside the main loop.

The main application function is divided into several smaller handler functions:

```
app_handle_buttons();  
app_handle_temperature();  
app_handle_fsm();  
app_handle_control();  
app_update_lcd();
```

Each handler is responsible for one specific task. This separation keeps the main loop easy to understand and prevents the application logic from becoming concentrated in one large function.

The overall operation of the software can be summarized as follows:

1. Initialize the LCD, DS18B20 sensor, buttons, fan driver, and application variables.
2. Repeatedly update the button states.
3. Process the temperature sensor state machine.
4. Execute the thermostat finite-state machine.
5. Apply the fan ON/OFF control algorithm.
6. Refresh the LCD display when required.

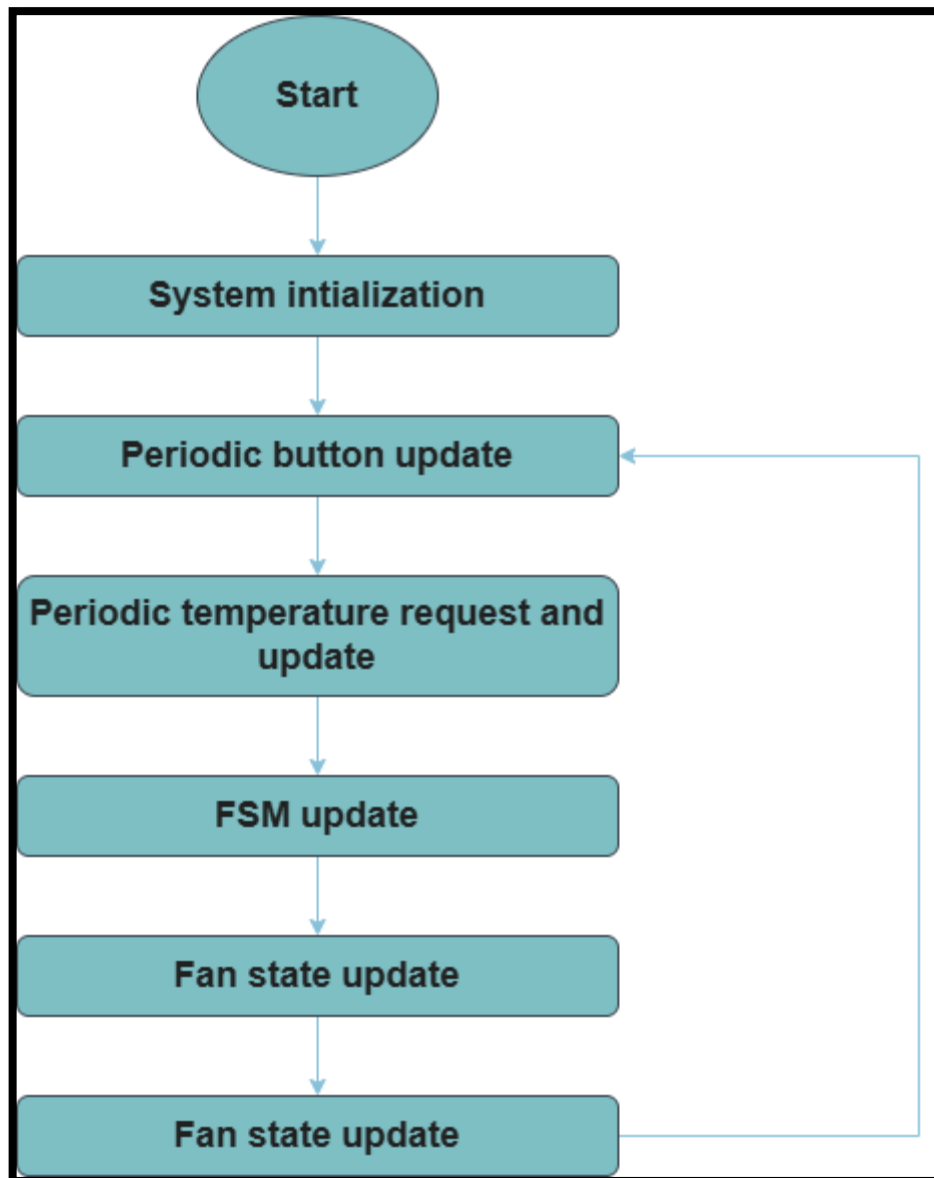


Figure 5.1: Main program flow of the thermostat software

5.2 System Initialization Flow

The application-level initialization is implemented in the function `app_main_init()`. This function prepares all peripherals and initializes the application variables before normal operation begins.

The initialization sequence is:

1. Initialize the LCD driver.
2. Initialize the DS18B20 driver.
3. Configure the DS18B20 to 11-bit resolution.

4. Clear the LCD and set the initial cursor position.
5. Initialize the UP, DOWN, SET, and POWER buttons.
6. Initialize the fan output.
7. Set the initial application state to normal mode.
8. Initialize the temperature variables and sensor-valid flag.

The system starts with:

Variable	Initial value	Meaning
app_state	APP_STATE_NORMAL	System begins in normal thermostat mode
temp_current	0	No valid temperature has been received yet
temp_edit	25	Initial editable setpoint
temp_set	25	Initial operating setpoint
sensor_valid	false	LCD update is disabled until a valid sensor reading is available

The DS18B20 is configured to 11-bit resolution during initialization. This provides a shorter conversion time than 12-bit mode while still being more than sufficient for the 1°C resolution required by the thermostat.

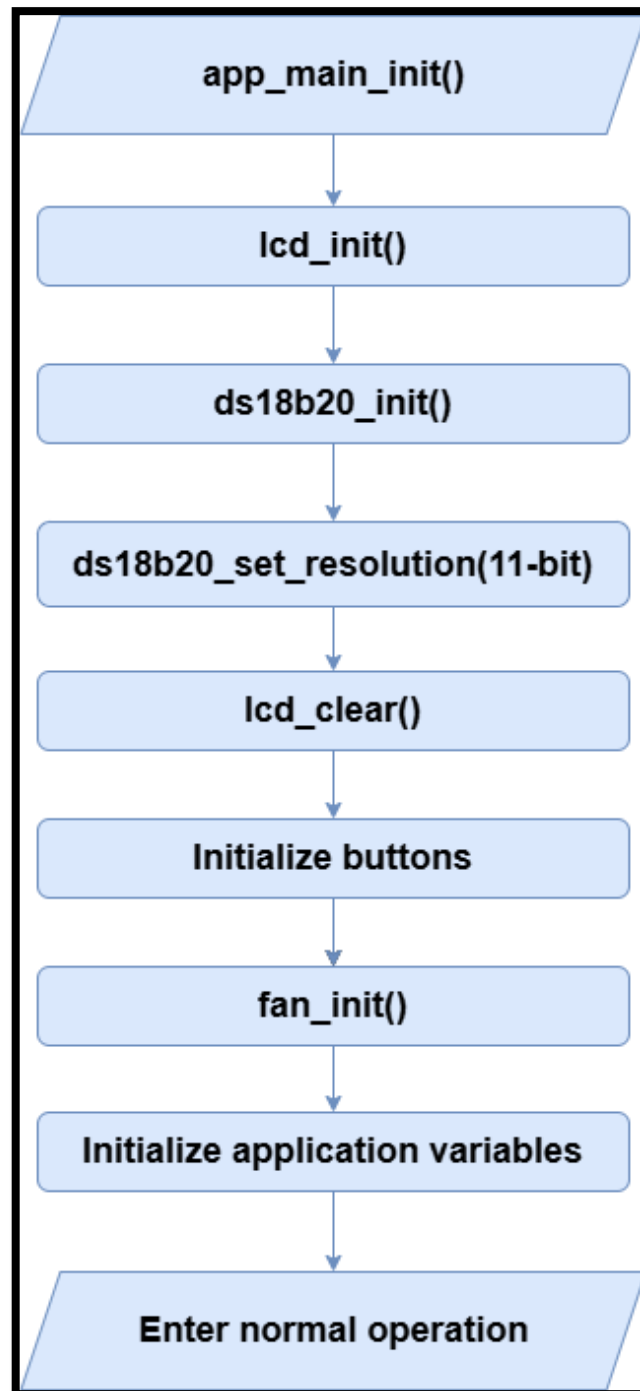


Figure 5.2: System initialization flow of the thermostat application

5.3 Super Loop Design

The thermostat uses a super-loop structure instead of a real-time operating system. In this design, the processor repeatedly executes the application tasks in a fixed order. Each task checks whether it is time to perform an action and returns immediately if no action is required.

The main super loop is represented by the function *app_main_run()*:

```

void app_main_run(void)
{
    app_handle_buttons();
    app_handle_temperature();
    app_handle_fsm();
    app_handle_control();
    app_update_lcd();
}

```

This design provides several advantages:

- It is simple and suitable for a small embedded application.
- It avoids the overhead of an operating system.
- It supports periodic task execution using timestamp checks.
- It allows the DS18B20 conversion to proceed without blocking the entire program.

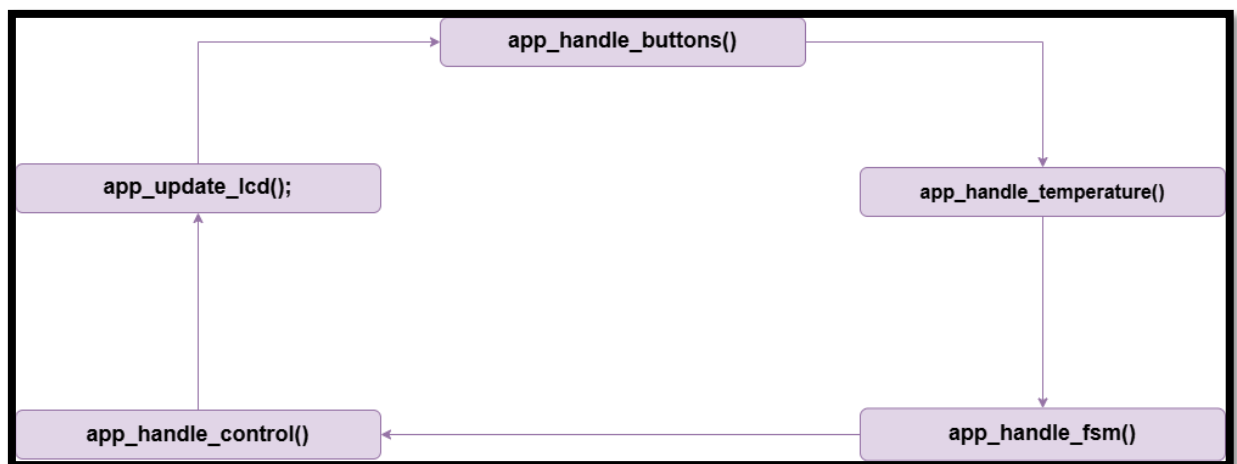


Figure 5.3: Super-loop execution flow of the thermostat firmware

5.4 Timing Design

The thermostat application relies on periodic timing rather than long blocking delays. A 1 ms SysTick counter acts as the global software time base. Each periodic task stores the time of its previous execution and compares it with the current tick value.

The main timing intervals used in the application are shown in the following table:

Task	Interval	Function
Button polling	10 ms	Update debounce logic and detect button events

Temperature request	500 ms	Start a new DS18B20 temperature conversion
LCD refresh	200 ms	Update displayed values and operating status
System tick	1 ms	Global timing base

Table 5.1. Periodic software tasks

5.4.1 SysTick-based Time Base

The timing base is implemented by the systick component. The function *systick_init()* configures the STM32 SysTick timer to generate an interrupt every 1 ms:

```
SysTick->LOAD = (system_core_clock_hz / 1000U) - 1U;
```

Each SysTick interrupt calls *systick_irq_handler()*, which increments the global millisecond counter `gu32TickMs`. The current time can then be obtained anywhere in the software using:

```
systick_get_tick()
```

Instead of checking for exact time values, the application uses subtraction:

```
if ((now - last_tick) >= period)
```

This method is suitable for periodic scheduling and also remains valid when the 32-bit tick counter eventually overflows.

5.4.2 500 ms Temperature Reading Period

The thermostat requests a new temperature sample every 500 ms. This behavior is implemented in *app_handle_temperature()* using the variable `last_temp_tick`.

```
if ((now - last_temp_tick) >= 500U)
{
    last_temp_tick = now;
    ds18b20_request_sample(&sensor, now);
}
```

The DS18B20 is configured to 11-bit resolution, which has a conversion time of approximately 375 ms. Therefore, the conversion is completed before the next 500 ms sampling request occurs.

The selected timing provides a balance between responsiveness and simplicity:

- The temperature is updated frequently enough for thermostat control.
- The DS18B20 conversion has sufficient time to finish.
- The processor remains free to handle button input and LCD updates while conversion is in progress.

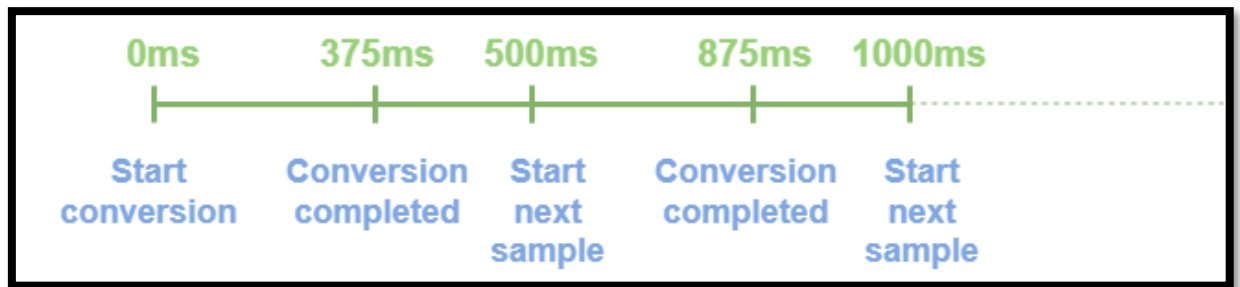


Figure 5.4: DS18B20 temperature sampling timeline

5.5 Button Handling Design

The push buttons are handled using periodic polling and software debouncing. The system uses four buttons:

- UP
- DOWN
- SET
- POWER

All buttons are configured as active-low inputs and are updated every 10 ms by *app_handle_buttons()*.

5.5.1 Button Functions

The button driver provides four main functions:

Function	Purpose
<i>button_init()</i>	Initializes the internal button state
<i>button_update()</i>	Reads the GPIO input and updates the debounce state
<i>button_is_pressed()</i>	Returns the current stable logical button state

<i>button_was_pressed()</i>	Returns a one-shot press event and then clears it
------------------------------------	---

The application mainly uses ***button_was_pressed()*** because a user action such as increasing the temperature should occur only once per press rather than continuously while the button is held.

5.5.2 Button Debouncing

Mechanical push buttons do not change directly from OFF to ON. When pressed or released, the contact may bounce several times within a short period. Without filtering, one physical press could be interpreted as multiple presses.

The button driver implements timestamp-based debouncing. Each button stores:

- Last raw input state,
- Stable debounced state,
- Last state-change timestamp,
- Debounce interval,
- One-shot press event flag.

When the raw GPIO level changes, the driver stores the new raw state and resets the debounce timer. Only if the raw state remains unchanged for at least 20 ms is it accepted as the new stable state. A press event is generated only when the stable state changes into the pressed condition.

The buttons are polled every 10 ms, while the debounce threshold is 20 ms. Therefore, a button state must remain stable across multiple polling cycles before it is accepted.

5.5.3 Setpoint Increase / Decrease Logic

The adjustable setpoint is modified only when the system is in APP_STATE_SETTING. When entering setting mode, the current operating setpoint is copied into the temporary variable temp_edit.

The UP button increases temp_edit by 1°C, while the DOWN button decreases it by 1°C. The allowable editing range is limited in software:

Condition	Action
temp_edit < 35 and UP pressed	Increase setpoint by 1°C
temp_edit > 15 and DOWN pressed	Decrease setpoint by 1°C

The temporary editing variable is used so that the user can adjust the desired value without immediately changing the active thermostat setpoint. When the SET button is pressed, temp_edit is copied into temp_set, and the system returns to normal mode.

This design separates the displayed editing value from the actual control value and prevents partial user input from affecting the fan control algorithm.

5.6 Temperature Reading Design

The DS18B20 sensor is used for digital temperature acquisition. The driver is implemented specifically for the project and handles both low-level 1-Wire communication and higher-level temperature processing.

5.6.1 DS18B20 Communication Overview

The DS18B20 uses a 1-Wire communication interface. A single data line is shared for both transmission and reception. In the thermostat implementation, the GPIO pin is dynamically switched between:

- open-drain output mode, when the microcontroller actively pulls the bus low,
- input mode with pull-up, when the line is released and the sensor or pull-up resistor determines the bus level.

The driver implements the required 1-Wire operations:

Operation	Function
Reset and presence detection	<i>ds18b20_reset()</i>
Write one bit	<i>ds18b20_write_bit()</i>
Read one bit	<i>ds18b20_read_bit()</i>
Write one byte	<i>ds18b20_write_byte()</i>
Read one byte	<i>ds18b20_read_byte()</i>

The main DS18B20 commands used by the driver are:

Command	Value	Purpose
SKIP ROM	0xCC	Address the only sensor on the bus
CONVERT T	0x44	Start temperature conversion
READ SCRATCHPAD	0xBE	Read the sensor memory
WRITE SCRATCHPAD	0x4E	Configure the resolution

The driver reads the complete 9-byte scratchpad and verifies the data using CRC-8 before accepting the measurement. A correct pull-up resistor on the data line is necessary for reliable operation. During development, the absence of a proper pull-up caused unstable readings even though the sensor could sometimes still be detected.

5.6.2 Temperature Conversion

After a conversion is completed, the driver reads the DS18B20 scratchpad. The first two bytes of the scratchpad contain the raw signed temperature value:

```
raw_temp = ((uint16_t)scratchpad[1] << 8U) | scratchpad[0];
```

```
raw_temp = ((uint16_t)scratchpad[1] << 8U) | scratchpad[0];
```

The raw value is then interpreted as a signed 16-bit number and converted into Celsius using:

```
temperature = raw_temp / 16.0
```

Because the thermostat only requires integer-degree control and display, the driver converts the raw temperature into a rounded integer value. Positive and negative values are rounded separately so that the rounding behavior remains correct for both cases.

The converted integer value is stored as `last_temp_int`. It becomes available to the application only after the scratchpad passes the CRC check.

5.6.3 Temperature Resolution Handling

The DS18B20 supports four different measurement resolutions. The driver selects the resolution by writing the configuration byte to the scratchpad.

Resolution	Configuration byte	Maximum conversion time
9-bit	0x1F	94 ms
10-bit	0x3F	188 ms
11-bit	0x5F	375 ms
12-bit	0x7F	750 ms

The application selects 11-bit resolution during initialization. This choice is appropriate because:

- The thermostat only requires 1°C display and control resolution,
- 11-bit mode offers more precision than required,
- The 375 ms conversion time fits comfortably within the 500 ms sampling period,

- It reduces waiting time compared with 12-bit mode.

5.6.4 Non-blocking Temperature Acquisition

A major design improvement of the project is the use of non-blocking temperature acquisition. This is achieved using an internal state machine

The driver uses two states:

State	Meaning
DS18B20_STATE_IDLE	No conversion is currently in progress
DS18B20_STATE_WAIT_CONVERSION	A conversion has started and the driver is waiting for the conversion time to elapse

When the application calls *ds18b20_request_sample()*, the driver starts a conversion, stores the start tick, and changes to WAIT_CONVERSION. The application then repeatedly calls *ds18b20_process()* during the super loop. Once the required conversion time has passed, the driver reads the scratchpad, checks the CRC, stores the latest temperature, and returns to IDLE.

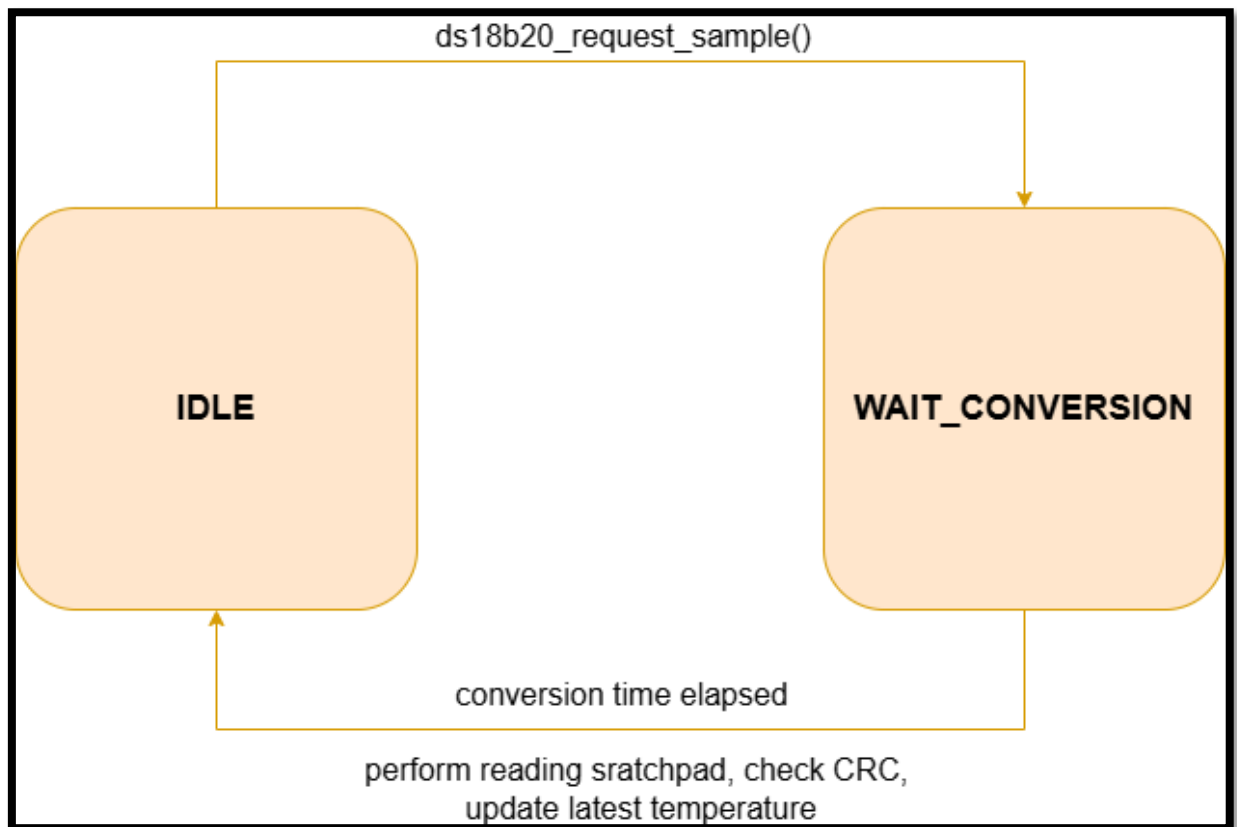


Figure 5.5: Non-blocking DS18B20 driver state machine

This design allows the thermostat to continue polling buttons, running the state machine, controlling the fan, and refreshing the LCD while the sensor is performing its internal conversion.

5.7 Fan Control Design

The thermostat uses a simple ON/OFF control method to drive the fan or cooling actuator. The active control algorithm is applied in normal mode and setting mode. In sleep mode and monitor mode, the fan is always turned off.

5.7.1 ON/OFF Control Algorithm

The control algorithm compares the current measured temperature `temp_current` with the selected setpoint `temp_set`.

```
if (temp_current > temp_set)
{
    fan_on(&fan);
}
else
{
    fan_off(&fan);
}
```

This is a direct ON/OFF controller without hysteresis. The method is simple and adequate for the required thermostat demonstration. If finer actuator behavior is required in future work, a hysteresis band could be added to reduce rapid switching near the setpoint.

5.7.2 Fan ON Condition

The fan is turned on when all of the following conditions are satisfied:

1. The system is in `APP_STATE_NORMAL` or `APP_STATE_SETTING`.
2. The measured temperature is greater than the selected setpoint:

This means the measured temperature is above the desired value and cooling is required.

5.7.3 Fan OFF Condition

The fan is turned off under any of the following conditions:

1. The measured temperature is less than or equal to the setpoint:
2. The system is in `APP_STATE_SLEEP`.
3. The system is in `APP_STATE_MONITOR`.
4. An undefined application state is encountered.

This behavior ensures that the fan remains inactive whenever the system is not performing automatic thermostat control.

5.8 LCD Display Design

The LCD provides the main user feedback interface of the thermostat. The application updates the display every 200 ms when a valid temperature reading is available.

5.8.1 Displayed Information

The displayed content depends on the current application state.

State	Displayed information
Normal	Current temperature, active setpoint, and fan status
Setting	Editable set temperature and current measured temperature
Monitor	Current temperature, AC disabled indication, and fan status
Sleep	LCD content cleared
Invalid sensor data	No display update performed until valid data is available

In normal mode, the first row displays the current temperature and the second row displays the active setpoint and fan state. In setting mode, the display shows the value being edited so the user can confirm the new setpoint before applying it. In monitor mode, the system still displays temperature information but indicates that automatic AC control is disabled.

5.8.2 LCD Update Strategy

LCD updates are performed periodically rather than continuously. The function *app_update_lcd()* checks whether at least 200 ms have passed since the previous update. If not, it returns immediately.

```
if (now - last_lcd_tick < 200U)
{
    return;
}
```

This strategy prevents unnecessary display writes and reduces flicker. The function also checks the *sensor_valid* flag before updating the display. Therefore, the LCD is not updated with uninitialized or invalid sensor data.

At the driver level, each command or data transfer includes the required timing delays. This is necessary because the HD44780 controller needs time to process instructions, especially after initialization and clear-display commands.

5.8.3 Fan Status Display

The current fan state is obtained from the fan driver using:

```
fan_get_state(&fan)
```

The application displays either *ON* or *OFF* depending on the cached fan state stored by the driver. This separates display logic from the low-level GPIO output operation and allows the LCD to show the actual software-controlled actuator state.

5.9 Thermostat State Machine Design

The application uses a finite-state machine to organize its operating modes. This makes the system behavior explicit and separates normal thermostat control from user configuration, sleep behavior, and temperature-only monitoring.

5.9.1 State Definition

The application defines four states.

State	Description
APP_STATE_NORMAL	Normal thermostat operation with automatic fan control
APP_STATE_SETTING	User edits the temperature setpoint
APP_STATE_SLEEP	Fan disabled and LCD cleared
APP_STATE_MONITOR	Temperature displayed while automatic fan control remains disabled

5.9.2 State Transition Diagram

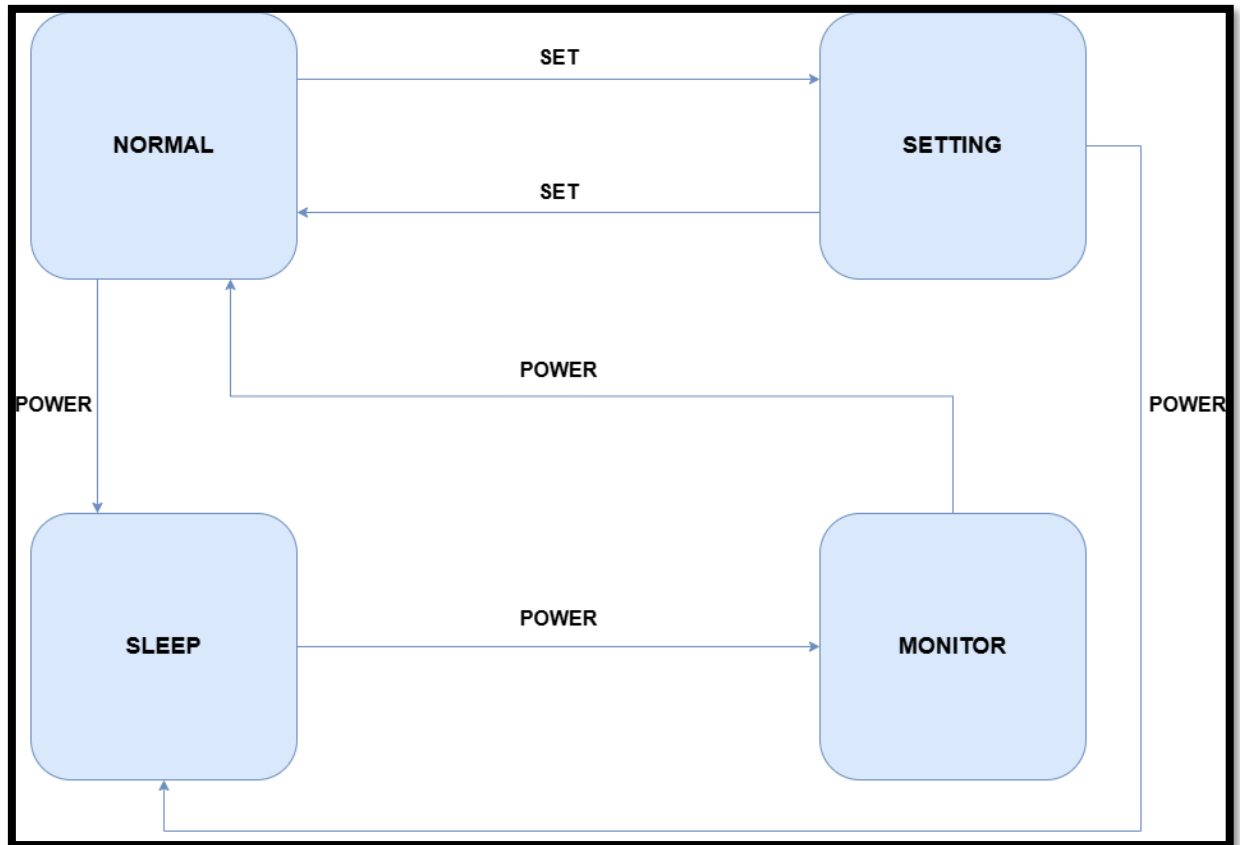


Figure 5.6: Thermostat application state transition diagram

5.9.3 State Transition Table

Current state	Event	Next state	Action
Normal	SET pressed	Setting	Copy temp_set into temp_edit
Normal	POWER pressed	Sleep	Disable fan through control logic
Setting	UP pressed	Setting	Increase temp_edit if below 35°C
Setting	DOWN pressed	Setting	Decrease temp_edit if above 15°C
Setting	SET pressed	Normal	Save temp_edit into temp_set
Setting	POWER pressed	Sleep	Leave setting mode and disable fan
Sleep	POWER pressed	Monitor	Resume temperature monitoring
Monitor	POWER pressed	Normal	Return to normal thermostat control

The finite-state-machine approach improves the clarity of the application logic. Each mode has a well-defined purpose, and the allowed transitions are controlled explicitly by button events.

7. Proteus Simulation

This chapter presents the Proteus simulation model used to verify the operation of the thermostat system before hardware implementation. The simulation reproduces the main functional blocks of the system, including temperature sensing, user input, LCD display, and cooling fan control. It is used to confirm that the firmware and peripheral connections operate correctly under different temperature and user-input conditions.

7.1 Simulation Circuit Overview

The Proteus simulation circuit is based on the same functional structure as the designed thermostat system. An STM32F401CE microcontroller model is used as the central controller. It receives temperature data from a DS18B20 digital sensor, reads four push-button inputs, updates a 16x2 LCD display, and controls a simulated cooling fan through a transistor-based driver circuit.

The schematic is divided into several functional blocks:

- DS18B20 temperature sensor interface
- STM32F401CE microcontroller
- LCD display interface
- Push-button input interface
- Cooling fan driver circuit

The simulation allows the system operation to be tested without requiring the physical hardware. By changing the temperature value of the DS18B20 model and pressing the simulated buttons, the control response, display output, and actuator behavior can be observed directly.

7.2 Proteus Schematic

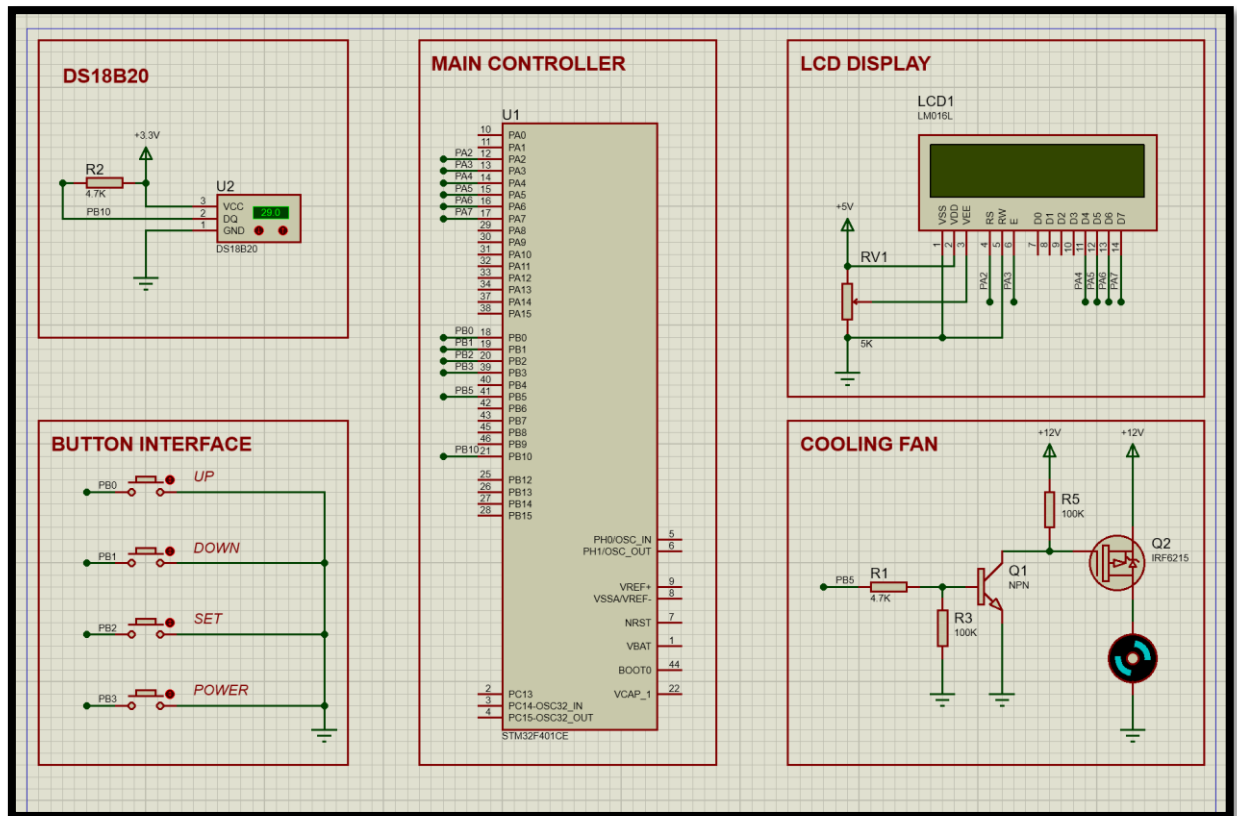


Figure 7.1: Complete Proteus schematic of the thermostat system

The DS18B20 data line is connected to PB10 of the microcontroller and pulled up to 3.3 V through a 4.7 k Ω resistor, which is required for proper 1-Wire communication. The LCD is connected in 4-bit mode, with control and data signals connected to GPIOA pins PA2 to PA7. The four push buttons are connected to PB0 to PB3 and operate as active-low inputs by pulling the input pins to ground when pressed.

The fan is driven from a 12 V supply through a switching circuit consisting of an NPN transistor and a P-channel MOSFET. The control signal from PB5 drives the base of the NPN transistor through a resistor. When PB5 is activated, the transistor pulls the MOSFET gate low, turning on the MOSFET and supplying power to the fan. This arrangement allows the STM32 output pin to control a higher-power load without driving the fan directly.

7.3 Simulation Components

The main components used in the Proteus simulation are summarized in Table 7.1.

Component	Role in the simulation
STM32F401CE	Executes the thermostat firmware and controls all peripherals

DS18B20	Simulates digital room-temperature measurement
4.7 k Ω pull-up resistor	Pulls the DS18B20 1-Wire data line to 3.3 V
16x2 LCD module	Displays temperature, setpoint, and system status
5 k Ω potentiometer	Adjusts LCD contrast voltage
Four push buttons	Provide UP, DOWN, SET, and POWER user inputs
NPN transistor	Interfaces the STM32 control signal to the MOSFET gate
P-channel MOSFET	Switches the 12 V fan supply
DC fan model	Represents the cooling actuator output

The simulation uses the same signal assignment as the implemented software design. The LCD uses PA2 to PA7, the DS18B20 uses PB10, the buttons use PB0 to PB3, and the fan control signal uses PB5.

7.4 Simulation Setup

The compiled firmware generated from the STM32 development environment is loaded into the STM32F401CE model in Proteus. After the simulation starts, the microcontroller executes the thermostat program and initializes the connected peripherals.

The DS18B20 model is used to provide adjustable temperature values during testing. The push buttons are operated manually in the simulation to verify setpoint adjustment and mode switching. The LCD output is observed to confirm that the displayed temperature, selected setpoint, and fan status correspond to the current system condition.

The simulation is used to verify the following functions:

1. communication between the STM32 and DS18B20 sensor;
2. correct LCD initialization and display update;
3. response of the UP, DOWN, SET, and POWER buttons;
4. ON/OFF control of the cooling fan output;
5. consistency between measured temperature, selected setpoint, and displayed system state.

7.5 Expected Simulation Behavior

When the simulation starts, the thermostat initializes the peripherals and displays the current temperature together with the selected setpoint and fan status. The DS18B20 sensor value can then be changed to test the cooling control behavior.

The expected operating behavior is summarized in following table

Test condition	Expected system response
System startup	LCD initializes and shows thermostat information after a valid temperature reading is available
Measured temperature higher than setpoint	Cooling fan turns on
Measured temperature equal to or lower than setpoint	Cooling fan turns off
SET button pressed	System enters or confirms setpoint setting mode
UP button pressed in setting mode	Editable setpoint increases
DOWN button pressed in setting mode	Editable setpoint decreases
POWER button pressed	System changes operating mode according to the application state machine
Temperature value changed in DS18B20 model	LCD and fan control update according to the new reading

The Proteus simulation therefore provides a convenient method for validating both the circuit connections and the embedded firmware behavior before practical deployment on physical hardware.

8. Testing and Results

8.1 Testing Methodology

The testing process was conducted to verify whether the implemented thermostat system satisfies the main requirements of the assignment. The system is required to monitor the room temperature from 0°C to 60°C, read temperature every 500 ms, provide a user interface for setting the temperature threshold, control the fan using an ON/OFF algorithm, display the ON/OFF state of the fan, and show the current temperature on the LCD during operation. The prototype is tested in Proteus simulation using an STM32 microcontroller, buttons, DS18B20 temperature sensor, LCD, and fan output.

The testing method follows a functional verification approach. Each major function of the system is tested separately first, and then the complete system behavior is tested after integration. The tested functions include system startup, temperature reading, setpoint adjustment, fan control, LCD update, and timing behavior.

The main purpose of testing is to confirm that the input devices, processing logic, and output devices work together correctly. The DS18B20 provides the current temperature value, the push buttons allow the user to adjust the setpoint, the application layer compares the current temperature with the setpoint, and the fan output is updated according to the ON/OFF control algorithm. The LCD is used to observe the current system state directly during simulation.

The testing process also considers the super-loop structure of the firmware. In this design, several tasks are called repeatedly inside the main loop, such as button handling, temperature reading, fan control, and LCD update. This follows the embedded programming approach introduced in the course, where multiple tasks can be executed in a super loop by calling their update functions continuously.

For the timing requirement, the temperature reading task is checked to make sure that the sensor value is updated every 500 ms. The course material explains that SysTick can be used to generate a regular time base and implement software timers, commonly with a 1 ms resolution. Therefore, the thermostat timing behavior can be verified by observing that the temperature update is executed after the required 500 ms interval.

The following criteria are used to determine whether a test case passes:

Criterion	Description
Correct input response	The system responds correctly when temperature or button inputs change.
Correct output behavior	The fan and LCD display change according to the expected logic.
Timing correctness	Temperature reading is updated with the required 500 ms period.
System stability	The system continues running in the main loop without freezing or resetting.
Requirement satisfaction	The behavior matches the functional and non-functional requirements.

8.2 Functional Test Cases

8.2.1 System Startup Test

The first test verifies the initial behavior of the system after reset or power-up. When the simulation starts, the STM32 should initialize all required peripherals, including GPIO pins, DS18B20 sensor interface, LCD display, button inputs, fan output, and timing components.

The expected behavior is that the LCD displays valid information after initialization. The fan should be OFF at startup unless the initial measured temperature is already higher than the set temperature. The setpoint should be initialized to a default value defined in the application software.

Table 8.1. System Startup Test

Item	Description
Test ID	TC1
Test name	System Startup Test
Input condition	Start Proteus simulation or reset the MCU
Expected result	LCD initializes correctly, default setpoint is displayed, fan starts in correct state
Observed result	LCD displayed the initial temperature and setpoint; fan output was initialized correctly
Status	Pass

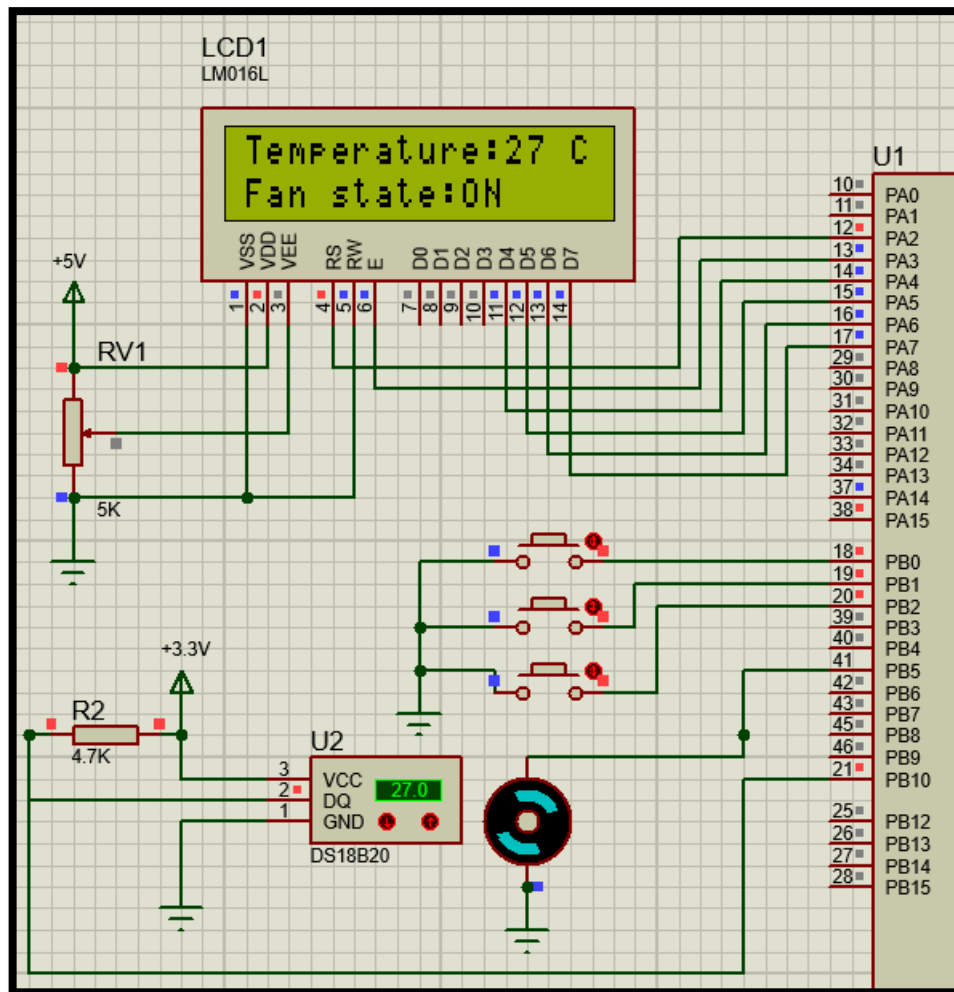


Figure 8.1. Initial system state after startup

8.2.2 Temperature Reading Test

This test verifies whether the system can read temperature data from the DS18B20 sensor. In Proteus, the DS18B20 temperature value can be changed manually. The LCD should update and show the current measured temperature.

The expected behavior is that when the DS18B20 temperature is changed, the value shown on the LCD also changes accordingly. Since the assignment requires 1°C resolution, the displayed temperature is treated as an integer value in degrees Celsius.

Table 8.2. Temperature Reading Test

Item	Description
Test ID	TC2
Test name	Temperature Reading Test

Input condition	Change DS18B20 temperature value in Proteus
Expected result	LCD displays the updated temperature value
Observed result	The LCD temperature value changed according to the DS18B20 setting
Status	Pass

Example observed cases:

DS18B20 Temperature	Expected LCD Temperature	Observed LCD Temperature	Result
25°C	25°C	25°C	Pass
28°C	28°C	28°C	Pass
32°C	32°C	32°C	Pass

8.2.3 Setpoint Adjustment Test

This test verifies the button-based user interface. The thermostat should allow the user to change the set temperature using push buttons. The UP button increases the setpoint, while the DOWN button decreases the setpoint. If a SET button is implemented, it can be used to enter or confirm the setting mode.

The expected behavior is that each valid button press changes the setpoint by one step. The LCD should reflect the new setpoint after the button is pressed. Button debouncing is also checked by observing whether one physical press results in only one setpoint change.

Table 8.3. Setpoint Adjustment Test

Item	Description
Test ID	TC3

Test name	Setpoint Adjustment Test
Input condition	Press UP or DOWN button
Expected result	Setpoint increases or decreases correctly
Observed result	The setpoint displayed on LCD changed correctly when buttons were pressed
Status	Pass

Example observed cases:

Action	Previous Setpoint	Expected Setpoint	Observed Setpoint	Result
Press UP	26°C	27°C	27°C	Pass
Press UP	27°C	28°C	28°C	Pass
Press DOWN	28°C	27°C	27°C	Pass
Press DOWN	27°C	26°C	26°C	Pass

8.2.4 Fan ON/OFF Control Test

This is the main control test of the thermostat system. The fan output should depend on the comparison between the current temperature and the set temperature. The ON/OFF algorithm is simple: when the current temperature is higher than the setpoint, the fan is turned ON; otherwise, the fan is turned OFF.

The test is performed by changing the DS18B20 temperature in Proteus while keeping the setpoint fixed. The fan output and LCD fan status are then observed.

Table 8.4. Fan ON/OFF Control Test

Item	Description
Test ID	TC4
Test name	Fan ON/OFF Control Test
Input condition	Change current temperature above or below setpoint
Expected result	Fan turns ON when current temperature is higher than setpoint; fan turns OFF when current temperature is lower than or equal to setpoint
Observed result	Fan output changed correctly according to the ON/OFF control logic
Status	Pass

Example observed cases:

Current Temperature	Setpoint	Expected Fan State	Observed Fan State	Result
25°C	28°C	OFF	OFF	Pass
28°C	28°C	OFF	OFF	Pass

30°C	28°C	ON	ON	Pass
32°C	28°C	ON	ON	Pass
27°C	28°C	OFF	OFF	Pass

The result shows that the thermostat control logic works as expected. When the room temperature is higher than the desired temperature, the fan is activated to simulate cooling. When the temperature is already at or below the setpoint, the fan remains OFF.

8.2.5 LCD Display Test

This test verifies whether the LCD provides correct feedback to the user. The LCD should display the current temperature, setpoint, and fan status. These values should change when the temperature or button input changes.

The test is performed by changing the sensor temperature and pressing the buttons during simulation. The LCD content is observed after each change.

Table 8.5. LCD Display Test

Item	Description
Test ID	TC5
Test name	LCD Display Test
Input condition	Change temperature, setpoint, and fan state
Expected result	LCD displays current temperature, setpoint, and fan ON/OFF status
Observed result	LCD updated the displayed temperature, setpoint, and fan state correctly
Status	Pass

A typical LCD output format is shown below:

Temp: 30 C Set: 28 C Fan: ON

When the temperature was reduced below the setpoint, the fan status displayed on the LCD changed from ON to OFF, showing that the display was synchronized with the control output.

8.2.6 500 ms Update Timing Test

Item	Description
Test ID	TC6
Test name	500 ms Update Timing Test
Input condition	Run simulation and observe periodic temperature update
Expected result	Temperature value is refreshed every 500 ms
Observed result	Temperature reading and LCD update followed the required update interval
Status	Pass

This result confirms that the thermostat does not update the sensor randomly. The timing behavior is consistent with the required 500 ms reading period stated in the assignment.

8.3 Test Result Table

The complete functional test results are summarized in Table 8.7.

Table 8.7. Summary of Functional Test Results

Test ID	Test Case	Expected Result	Actual Result	Status

TC1	System Startup Test	System initializes correctly; LCD and fan show correct initial state	LCD initialized and fan state was correct	Pass
TC2	Temperature Reading Test	Current temperature is read from DS18B20 and displayed on LCD	LCD showed the updated sensor temperature	Pass
TC3	Setpoint Adjustment Test	UP/DOWN buttons change the setpoint correctly	Setpoint changed correctly on LCD	Pass
TC4	Fan ON/OFF Control Test	Fan turns ON/OFF based on temperature and setpoint comparison	Fan output followed the ON/OFF control logic	Pass
TC5	LCD Display Test	LCD shows current temperature, setpoint, and fan status	LCD displayed all required information	Pass
TC6	500 ms Update Timing Test	Temperature reading is updated every 500 ms	Temperature update followed the required timing	Pass

Based on the test results, all major functional requirements were satisfied in the Proteus simulation. The system successfully integrated the sensor input, button interface, control algorithm, fan output, and LCD display.

8.4 Simulation Screenshots

8.4.1 Initial System State

At the beginning of the simulation, the STM32 initializes all modules and displays the initial system information on the LCD. The fan is initialized according to the current temperature and default setpoint.

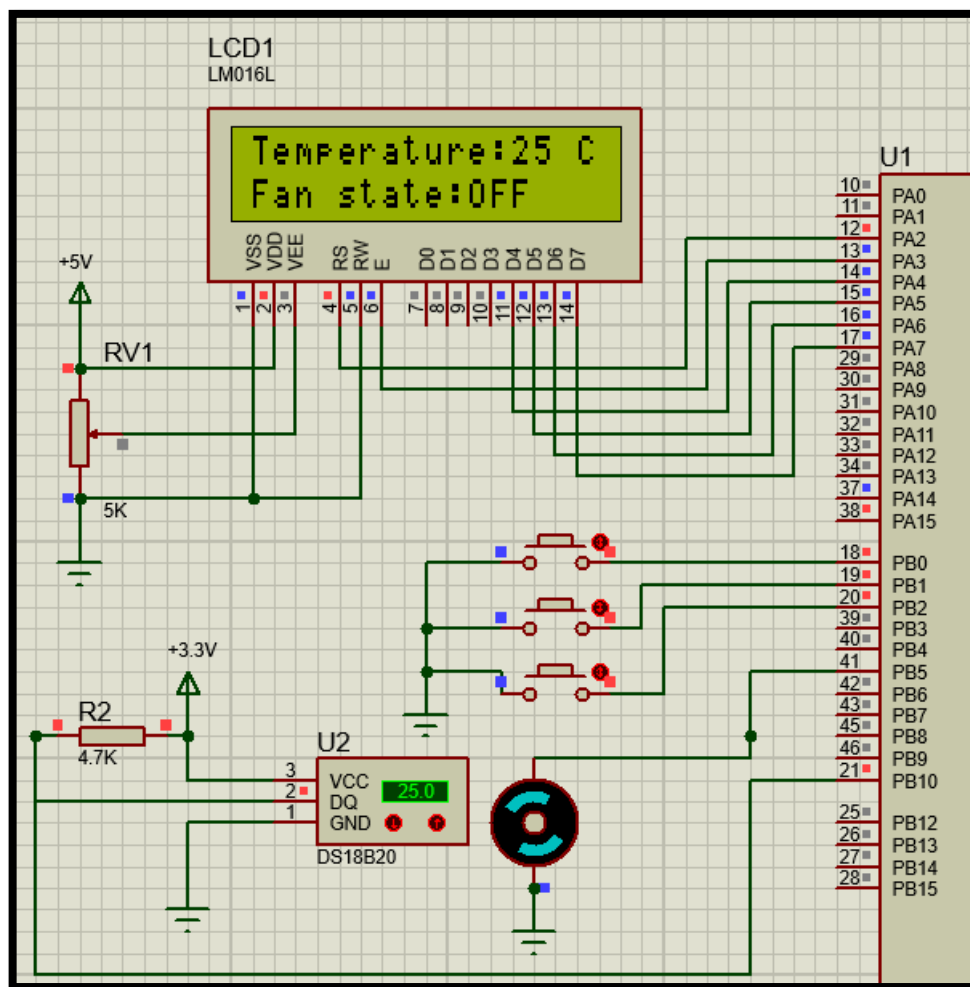


Figure 8.2. Initial system state in Proteus

Expected display example:

Temp: 25 C
Set: 28 C Fan: OFF

This screenshot should show that the LCD is working and that the system has entered normal operating mode after initialization.

8.4.2 Temperature Below Setpoint

In this test condition, the current temperature is lower than the setpoint. Therefore, the cooling fan should remain OFF.

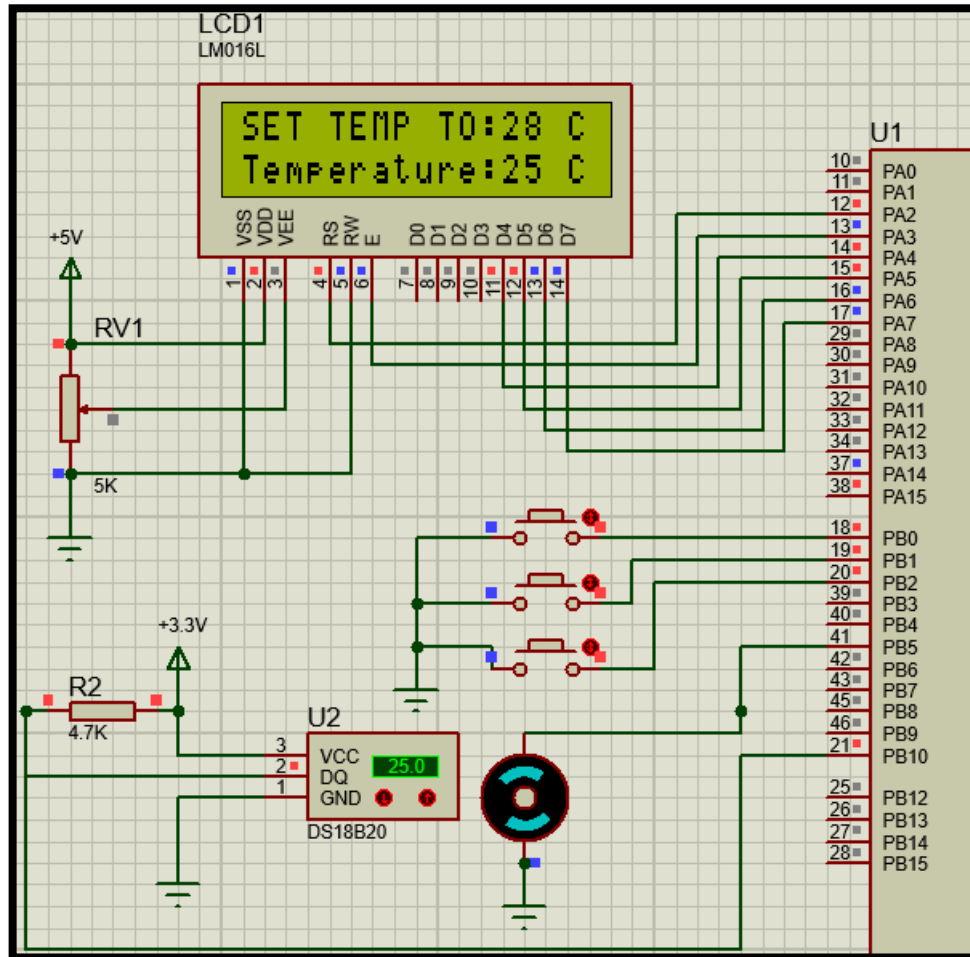


Figure 8.3. Temperature below setpoint

Example condition:

Current Temperature	Setpoint	Expected Fan State
25°C	28°C	OFF

Expected display example:

Temp: 25 C
Set: 28 C Fan: OFF

The result confirms that the fan is not activated unnecessarily when the room temperature is already below the desired value.

8.4.3 Temperature Above Setpoint

In this test condition, the current temperature is higher than the setpoint. Therefore, the thermostat should turn ON the fan.

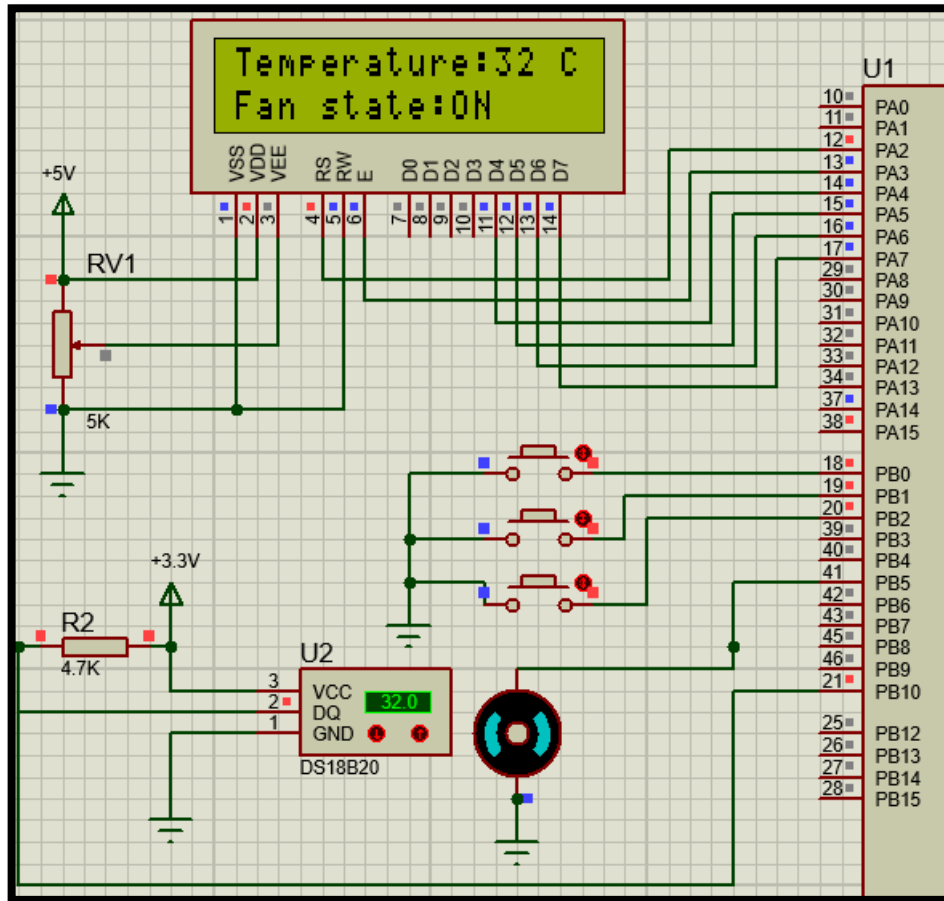


Figure 8.4. Temperature above setpoint

Example condition:

Current Temperature	Setpoint	Expected Fan State
32°C	28°C	ON

Expected display example:

Temp: 32 C
Set: 28 C Fan: ON

The result confirms that the thermostat can detect an over-temperature condition and activate the cooling output.

8.4.4 Fan ON State

This screenshot focuses on the fan output when the system is in the ON state. The fan output pin should be active, and the LCD should display Fan: ON.

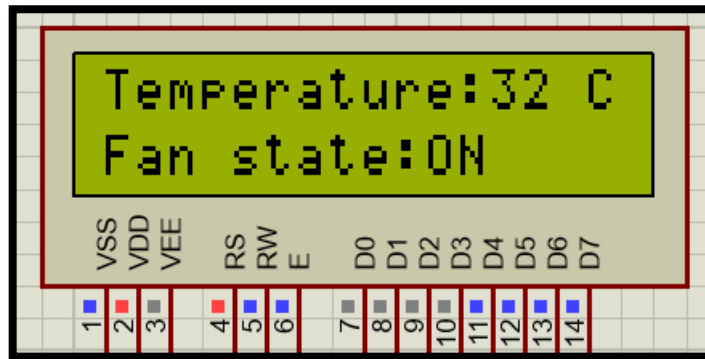


Figure 8.5. Fan ON state

The fan ON state occurs when:

Current temperature > Set temperature

This verifies that the software control signal and user display are consistent.

8.4.5 Fan OFF State

This screenshot shows the system when the fan is OFF. The fan output pin should be inactive, and the LCD should display Fan: OFF.

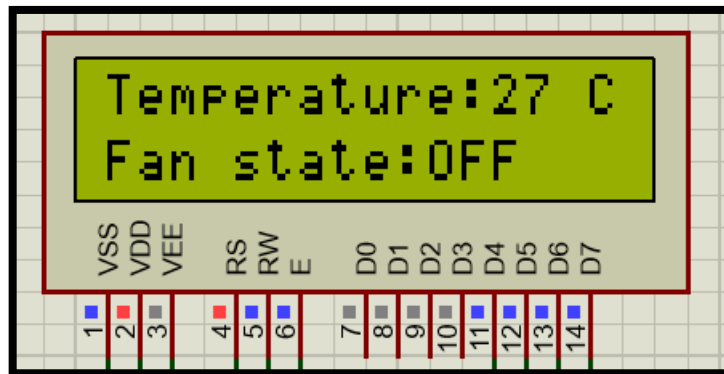


Figure 8.6. Fan OFF state

The fan OFF state occurs when:

Current temperature <= Set temperature

This verifies that the thermostat stops the fan when cooling is not required.

8.4.6 Setpoint Adjustment using Buttons

This test shows the LCD response when the user presses the UP or DOWN button. The setpoint should change by one degree for each valid button press.

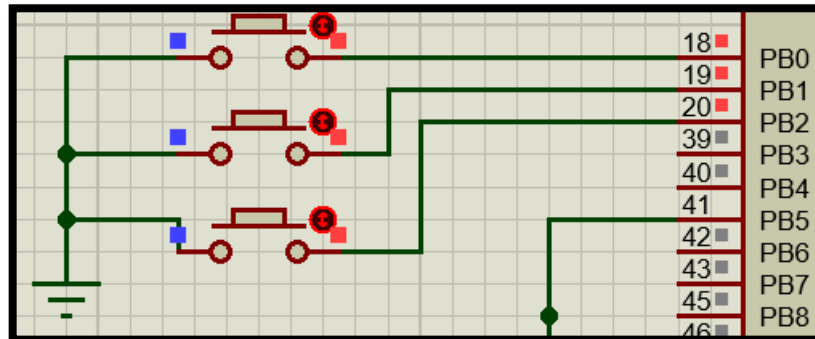


Figure 8.7. Setpoint adjustment using buttons

Example sequence:

Action	LCD Before	LCD After
Press UP	Set: 28 C	Set: 29 C
Press DOWN	Set: 29 C	Set: 28 C

The result confirms that the button-based user interface works correctly and that the LCD updates after setpoint changes.

8.5 Requirement Verification Matrix

The requirement verification matrix is used to confirm that each requirement from the assignment has been tested and satisfied by the implemented system.

Table 8.8. Requirement Verification Matrix

Requirement ID	Requirement	Related Test Case	Verification Method	Result
FR1	Monitor room temperature from 0°C to 60°C	TC2	Change DS18B20 temperature and observe LCD	Pass

FR2	Read temperature every 500 ms	TC6	Observe periodic temperature update	Pass
FR3	Temperature resolution of 1°C	TC2, TC5	Check integer temperature display on LCD	Pass
FR4	Provide user interface for setting temperature threshold	TC3	Press UP/DOWN buttons and observe setpoint	Pass
FR5	Control fan using ON/OFF algorithm	TC4	Compare fan state with temperature and setpoint	Pass
FR6	Display fan ON/OFF status	TC5	Observe fan status on LCD	Pass
FR7	Display current temperature on LCD	TC2, TC5	Observe current temperature on LCD	Pass
FR8	Proteus prototype with STM32, buttons, DS18B20, and LCD	TC1–TC6	Run integrated Proteus simulation	Pass

The verification matrix shows that the implemented thermostat satisfies the core assignment requirements. The system reads temperature periodically, allows the user to

adjust the setpoint, controls the fan using ON/OFF logic, and displays the required information on the LCD.

Overall, the testing results demonstrate that the thermostat system operates correctly in Proteus simulation. The integration between the application logic and the BSP modules is successful. The sensor input, button input, LCD output, and fan output work together as a complete embedded system.

9. Discussion

9.1 Achieved Results

The testing results show that the implemented thermostat system achieved the main functional goals of the project. The most important result is the successful integration of sensing, user input, display, timing, and fan control into one working embedded application. Instead of testing each peripheral separately, the final system was able to coordinate all modules through the application layer.

The modular software structure also proved to be effective during implementation and testing. The application layer handled the thermostat behavior, while the BSP and driver layers isolated hardware-specific details. This separation made the firmware easier to debug and modify when testing the DS18B20 sensor, LCD, buttons, and fan output.

Another achieved result is the stable operation of the super-loop design. Button handling, temperature acquisition, LCD update, and fan control were executed repeatedly without requiring an operating system. The 500 ms temperature update period and the 200 ms LCD refresh period were sufficient for the simulated thermostat behavior.

Overall, the project demonstrates that a simple bare-metal STM32 application can be organized into clear modules and integrated into a complete temperature control system.

9.2 Limitations of the Current System

Although the system satisfies the main requirements, several limitations still remain.

First, the control algorithm uses a direct ON/OFF method without hysteresis. When the measured temperature fluctuates around the setpoint, the fan may switch ON and OFF frequently. This behavior is acceptable for demonstration, but it may reduce stability in a real cooling system.

Second, the system was mainly verified in Proteus simulation. Simulation is useful for checking logic behavior and circuit connections, but it cannot fully represent the physical behavior of real hardware, such as sensor delay, fan inertia, power noise, and actual room temperature dynamics.

Third, the current implementation has limited sensor error handling. If the DS18B20 is disconnected or returns invalid data, the system may not provide a clear warning to the user. A practical thermostat should detect this condition and enter a safe operating state.

Fourth, the selected setpoint is not stored permanently. After reset or power loss, the system returns to the default setpoint. This is acceptable for a prototype, but it reduces convenience in a real product.

9.3 Possible Improvements

Several improvements can be made to increase the reliability, usability, and scalability of the thermostat system.

9.3.1 Hysteresis Control

The first improvement is to add hysteresis to the ON/OFF control algorithm. Hysteresis prevents the fan from switching too frequently when the temperature is close to the setpoint.

Instead of using only one threshold, the controller can use two thresholds: one threshold to turn the fan ON and another threshold to turn it OFF. For example, if the setpoint is 28°C and the hysteresis value is 1°C, the fan can be turned ON when the temperature is greater than or equal to 29°C and turned OFF when the temperature is less than or equal to 28°C.

The improved logic can be described as follows:

If fan is OFF and current temperature $\geq$ setpoint + hysteresis:
Turn fan ON
If fan is ON and current temperature $\leq$ setpoint:
Turn fan OFF

This method reduces rapid ON/OFF switching and makes the system more stable. It is also closer to how real thermostats operate.

9.3.2 UART Debugging

Another useful improvement is adding UART debugging. UART can be used to send internal system information to a computer terminal during development and testing.

The system can print useful information such as:

Current temperature: 30 C
Setpoint: 28 C
Fan state: ON
Button UP pressed

Button DOWN pressed Sensor read success
--

This feature would make debugging easier because the developer can observe internal variables without relying only on the LCD. UART debugging is especially useful when the LCD display is limited to two lines or when the system has unexpected behavior.

UART can also be used later for communication with external devices, such as a Bluetooth module, Wi-Fi module, or PC-based monitoring software.

9.3.3 Sensor Error Detection

The system can be improved by adding sensor error detection. Since the DS18B20 is the main input of the thermostat, sensor failure can cause incorrect control behavior. Therefore, the firmware should check whether the sensor responds correctly before using the measured temperature.

Possible sensor error conditions include:

Error Condition	Possible System Response
DS18B20 does not respond	Display Sensor Error on LCD
Temperature value is outside valid range	Ignore value and keep previous valid reading
Sensor communication fails repeatedly	Turn fan OFF for safety
Invalid CRC data	Request a new temperature conversion

With this improvement, the thermostat would become safer and more reliable. If the sensor fails, the system should not continue controlling the fan based on invalid temperature data.

9.3.4 Saving Setpoint to Flash Memory

Another improvement is to save the user-defined setpoint into the STM32 internal Flash memory. In the current version, the setpoint is reset to a default value whenever the system restarts. By storing the setpoint in Flash, the thermostat can remember the last user setting even after power loss.

The behavior can be designed as follows:

When the user changes and confirms the setpoint:
Save the new setpoint to Flash memory

When the system starts:
Read the saved setpoint from Flash memory
If the saved value is valid:
Use it as the initial setpoint
Else:
Use the default setpoint

This feature improves user convenience because the user does not need to adjust the setpoint again after every reset.

However, Flash memory has a limited number of write cycles. Therefore, the firmware should avoid writing to Flash too frequently. For example, the system should only save the setpoint when the user confirms the setting, not every time the button is pressed.

9.3.5 RTOS-based Extension

For the current project, a super-loop design is enough because the system only has a few tasks. However, if the system is expanded, an RTOS-based design can be considered.

With an RTOS, the system can be divided into separate tasks such as:

RTOS Task	Responsibility
Temperature Task	Read DS18B20 every 500 ms
Button Task	Scan and debounce buttons
Control Task	Execute thermostat ON/OFF control
LCD Task	Update displayed information
Communication Task	Send debug or monitoring data through UART

This structure can make the system easier to extend and manage when more features are added. For example, UART communication, buzzer alarm, multiple sensors, or remote monitoring can be implemented as separate tasks.

However, using an RTOS also increases software complexity. It requires understanding task scheduling, priorities, stack size, shared resources, and synchronization mechanisms. Therefore, for this assignment, the super-loop structure is still a suitable and reasonable choice.

10. Conclusion

10.1 Project Summary

This project successfully designed and implemented an STM32-based thermostat controller. The system reads temperature from a DS18B20 sensor, allows setpoint adjustment through push buttons, controls a fan using an ON/OFF algorithm, and displays system information on an LCD. The prototype was verified in Proteus simulation and satisfied the main functional requirements of the assignment.

The firmware was organized using a modular structure with separate application, BSP, and driver/component layers. This organization helped separate thermostat logic from hardware-specific operations and made the system easier to implement, test, and maintain.

10.2 Learning Outcomes

Through this project, the group gained practical experience in STM32 embedded programming. The implementation required GPIO configuration, DS18B20 communication, LCD control, button handling, fan output control, and timing management using a SysTick-based time base.

The project also improved the understanding of modular firmware design. By separating the software into application, BSP, and driver/component layers, each module had a clear responsibility. This made the code more readable and easier to debug during integration.

In addition, the project provided experience in simulation-based testing. Proteus was used to verify the interaction between the firmware and the simulated hardware components. This helped the group check temperature reading, setpoint adjustment, LCD display, and fan control before moving to physical hardware.

10.3 Final Remarks

In conclusion, the mini-project achieved its goal of building a functional embedded thermostat prototype using STM32. The system combines sensor input, user interaction, timing control, display output, and actuator control into one complete application. Although the current implementation is still a basic prototype, it provides a solid foundation for future improvements and demonstrates the practical application of the concepts learned in the Embedded System Programming course.

References

- [1] STMicroelectronics, “STM32F401xD STM32F401xE Arm® Cortex®-M4 32-bit MCU+FPU datasheet,” DS10086, 2026.
- [2] STMicroelectronics, “RM0368 Reference manual: STM32F401xB/C and STM32F401xD/E advanced Arm®-based 32-bit MCUs,” Rev. 6, Jan. 2025.
- [3] STMicroelectronics, “PM0214 Programming manual: STM32 Cortex®-M4 MCUs and MPUs programming manual,” Rev. 10, Mar. 2020.
- [4] Arm Ltd., “CMSIS: Common Microcontroller Software Interface Standard documentation,” CMSIS 6 Documentation.
- [5] STMicroelectronics, “UM1725 User manual: Description of STM32F4 HAL and low-layer drivers,” User Manual.
- [6] STMicroelectronics, “UM2553 User manual: STM32CubeIDE quick start guide,” User Manual, 2026.
- [7] STMicroelectronics, “STM32CubeMX: STM32 configuration and initialization C code generation,” Development Tool Documentation.
- [8] STMicroelectronics, “AN4899 Application note: STM32 microcontroller GPIO hardware settings and low-power consumption,” Rev. 3, Mar. 2022.
- [9] Analog Devices, “DS18B20 Programmable Resolution 1-Wire Digital Thermometer Datasheet,” Data Sheet.
- [10] Hitachi, “HD44780U LCD-II Dot Matrix Liquid Crystal Display Controller/Driver Datasheet,” Rev. 0.0, Sept. 1999.
- [11] Labcenter Electronics, “Proteus Design Suite: PCB design and circuit simulation software,” Product Documentation.
- [12] B. Q. Bảo, “Lecture Slides for Embedded System Programming,” Ho Chi Minh City University of Technology, Vietnam National University Ho Chi Minh City, 2026.

Appendix

Appendix A. Full Project Folder Structure

The following structure summarizes the STM32CubeIDE project organization used for the thermostat firmware. Generated build-output files are listed separately from the custom source code to make the project easier to read.

```

Thermostat/
|-- Binaries/
|-- Includes/
|-- Core/
|   |-- Inc/
|   |   |-- main.h
|   |   |-- stm32f4xx_it.h
|   |   |-- stm32f4xx_hal_conf.h
|   |   |-- system_stm32f4xx.h
|   |-- Src/
|   |   |-- main.c
|   |   |-- stm32f4xx_it.c
|   |   |-- stm32f4xx_hal_msp.c
|   |   |-- syscalls.c
|   |   |-- systemem.c
|   |   |-- system_stm32f4xx.c
|-- Drivers/
|   |-- CMSIS/
|   |-- STM32F4xx_HAL_Driver/
|-- Src/
|   |-- App/
|   |   |-- app_main.c
|   |   |-- app_main.h
|   |-- BSP/
|   |   |-- BSP_Button/
|   |   |   |-- bsp_button.c
|   |   |   |-- bsp_button.h
|   |   |-- BSP_DS18B20/
|   |   |   |-- bsp_ds18b20.c
|   |   |   |-- bsp_ds18b20.h
|   |   |-- BSP_Fan/
|   |   |   |-- bsp_fan.c
|   |   |   |-- bsp_fan.h
|   |   |-- BSP_LCD/
|   |   |   |-- bsp_lcd.c
|   |   |   |-- bsp_lcd.h
|   |-- Drivers/
|   |   |-- Components/
|   |   |   |-- delay/
|   |   |   |-- dio/
|   |   |   |-- systick/
|   |   |-- Devices/
|   |   |   |-- button/
|   |   |   |-- ds18b20/
|   |   |   |-- fan/
|   |   |   |-- lcd/
|-- Debug/
|   |-- Thermostat.elf
|   |-- Thermostat.hex
|   |-- Thermostat.map
|   |-- other generated build files
|-- STM32F401CEUX_FLASH.ld
|-- STM32F401CEUX_RAM.ld
`-- Thermostat.ioc

```

Folder / File	Description
Core/	STM32CubeIDE-generated initialization files, interrupt handlers, and system-level source files.
Src/App/	High-level thermostat application logic and system state handling.

Src/BSP/	Board-specific pin mapping and binding between application modules and physical GPIO pins.
Src/Drivers/Components/	Reusable support modules such as delay, digital I/O abstraction, and SysTick timing base.
Src/Drivers/Devices/	Reusable device-level drivers for button, DS18B20, fan, and LCD modules.
Thermostat.ioc	STM32CubeMX configuration file for clock, GPIO, and project settings.
Debug/	Generated build output directory. It is not part of the custom firmware logic.

Appendix B. Full Pin Assignment Table

Peripheral	Signal	STM32 Port/Pin	Direction	Description
LCD	RS	PA2	Output	Register-select signal for LCD command/data selection.
LCD	EN	PA3	Output	Enable signal used to latch LCD data.
LCD	D4	PA4	Output	LCD 4-bit data line.
LCD	D5	PA5	Output	LCD 4-bit data line.
LCD	D6	PA6	Output	LCD 4-bit data line.

LCD	D7	PA7	Output	LCD 4-bit data line.
DS18B20	DQ	PB10	Input/Output	1-Wire data line with external pull-up resistor.
Push Button	UP	PB0	Input	Increases the editable setpoint in setting mode.
Push Button	DOWN	PB1	Input	Decreases the editable setpoint in setting mode.
Push Button	SET	PB2	Input	Enters setting mode or confirms the edited setpoint.
Push Button	POWER	PB3	Input	Switches between system operating modes.
Fan	FAN_CONTROL	PB5	Output	Controls the fan or cooling actuator through the driver circuit.

Appendix C. Important Source Code

C.1 Application Entry Points

```

/* main.c */
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    systick_init(SystemCoreClock);

```

```

delay_init(SystemCoreClock);
app_main_init();

while (1)
{
    app_main_run();
}

```

C.2 Main Super Loop

```

/* app_main.c */
void app_main_run(void)
{
    app_handle_buttons();
    app_handle_temperature();
    app_handle_fsm();
    app_handle_control();
    app_update_lcd();
}

```

C.3 Application Initialization

```

void app_main_init(void)
{
    bsp_lcd_init();
    bsp_ds18b20_init();
    bsp_ds18b20_set_resolution(DS18B20_RESOLUTION_11BIT);

    bsp_lcd_clear();
    bsp_button_init();
    bsp_fan_init();

    app_state = APP_STATE_NORMAL;
    temp_current = 0;
    temp_edit = 25;
    temp_set = 25;
    sensor_valid = false;
}

```

```

    bsp_fan_off();
}

```

C.4 SysTick Time Base

```

/* systick.c */
static volatile uint32_t gu32TickMs = 0;

void systick_init(uint32_t system_core_clock_hz)
{
    SysTick->LOAD = (system_core_clock_hz / 1000U) - 1U;
    SysTick->VAL = 0U;
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk |
                    SysTick_CTRL_ENABLE_Msk;
}

void systick_irq_handler(void)
{
    gu32TickMs++;
}

uint32_t systick_get_tick(void)
{
    return gu32TickMs;
}

```

C.5 Button Handling and Setpoint Editing

```

static void app_handle_buttons(void)
{
    static uint32_t last_button_tick = 0U;
    uint32_t now = systick_get_tick();

    if ((now - last_button_tick) < 10U)
    {
        return;
    }
}

```

```

    }
    last_button_tick = now;

    bsp_button_update(now);
}

static void app_handle_fsm(void)
{
    switch (app_state)
    {
        case APP_STATE_NORMAL:
            if (bsp_button_set_was_pressed())
            {
                temp_edit = temp_set;
                app_state = APP_STATE_SETTING;
            }
            else if (bsp_button_power_was_pressed())
            {
                app_state = APP_STATE_SLEEP;
            }
            break;

        case APP_STATE_SETTING:
            if (bsp_button_up_was_pressed() && temp_edit < 35)
            {
                temp_edit++;
            }
            else if (bsp_button_down_was_pressed() && temp_edit > 15)
            {
                temp_edit--;
            }
            else if (bsp_button_set_was_pressed())
            {
                temp_set = temp_edit;
                app_state = APP_STATE_NORMAL;
            }
            else if (bsp_button_power_was_pressed())
            {
                app_state = APP_STATE_SLEEP;
            }
            break;
    }
}

```

```

case APP_STATE_SLEEP:
    if (bsp_button_power_was_pressed())
    {
        app_state = APP_STATE_MONITOR;
    }
    break;

case APP_STATE_MONITOR:
    if (bsp_button_power_was_pressed())
    {
        app_state = APP_STATE_NORMAL;
    }
    break;

default:
    app_state = APP_STATE_NORMAL;
    break;
}
}

```

C.6 Periodic DS18B20 Temperature Acquisition

```

static void app_handle_temperature(void)
{
    static uint32_t last_temp_tick = 0U;
    uint32_t now = systick_get_tick();

    if ((now - last_temp_tick) >= 500U)
    {
        last_temp_tick = now;
        ds18b20_request_sample(&sensor, now);
    }

    ds18b20_process(&sensor, now);

    if (ds18b20_has_new_sample(&sensor))
    {
        temp_current = ds18b20_get_temperature_int(&sensor);
        sensor_valid = true;
    }
}

```

```
}  
}
```

C.7 Fan ON/OFF Control

```
static void app_handle_control(void)  
{  
    if (!sensor_valid)  
    {  
        bsp_fan_off();  
        return;  
    }  
  
    switch (app_state)  
    {  
        case APP_STATE_NORMAL:  
        case APP_STATE_SETTING:  
            if (temp_current > temp_set)  
            {  
                bsp_fan_on();  
            }  
            else  
            {  
                bsp_fan_off();  
            }  
            break;  
  
        case APP_STATE_SLEEP:  
        case APP_STATE_MONITOR:  
        default:  
            bsp_fan_off();  
            break;  
    }  
}
```

C.8 LCD Update

```
static void app_update_lcd(void)
```

```

{
    static uint32_t last_lcd_tick = 0U;
    uint32_t now = systick_get_tick();

    if ((now - last_lcd_tick) < 200U)
    {
        return;
    }
    last_lcd_tick = now;

    if (!sensor_valid)
    {
        return;
    }

    switch (app_state)
    {
    case APP_STATE_NORMAL:
        bsp_lcd_set_cursor(0, 0);
        bsp_lcd_printf("Temp:%2d C", temp_current);
        bsp_lcd_set_cursor(1, 0);
        bsp_lcd_printf("Set:%2d Fan:%s", temp_set,
                        bsp_fan_get_state() ? "ON " : "OFF");
        break;

    case APP_STATE_SETTING:
        bsp_lcd_set_cursor(0, 0);
        bsp_lcd_printf("Set Temp:%2d C", temp_edit);
        bsp_lcd_set_cursor(1, 0);
        bsp_lcd_printf("Current:%2d C", temp_current);
        break;

    case APP_STATE_MONITOR:
        bsp_lcd_set_cursor(0, 0);
        bsp_lcd_printf("Temp:%2d C", temp_current);
        bsp_lcd_set_cursor(1, 0);
        bsp_lcd_printf("Fan Control OFF");
        break;

    case APP_STATE_SLEEP:
        bsp_lcd_clear();
        break;
    }
}

```

```

    default:
        break;
    }
}

```

C.9 DS18B20 Non-blocking Driver State Machine

```

typedef enum
{
    DS18B20_STATE_IDLE = 0,
    DS18B20_STATE_WAIT_CONVERSION
} ds18b20_state_t;

void ds18b20_request_sample(ds18b20_t *dev, uint32_t now)
{
    if (dev->state != DS18B20_STATE_IDLE)
    {
        return;
    }

    ds18b20_reset(dev);
    ds18b20_write_byte(dev, DS18B20_CMD_SKIP_ROM);
    ds18b20_write_byte(dev, DS18B20_CMD_CONVERT_T);

    dev->start_tick = now;
    dev->state = DS18B20_STATE_WAIT_CONVERSION;
}

void ds18b20_process(ds18b20_t *dev, uint32_t now)
{
    if (dev->state != DS18B20_STATE_WAIT_CONVERSION)
    {
        return;
    }

    if ((now - dev->start_tick) < dev->conversion_time_ms)
    {
        return;
    }
}

```

```
if (ds18b20_read_scratchpad(dev))
{
    dev->last_temp_int = ds18b20_convert_to_integer(dev->scratchpad);
    dev->new_sample = true;
}

dev->state = DS18B20_STATE_IDLE;
}
```

Appendix D. Proteus Schematic

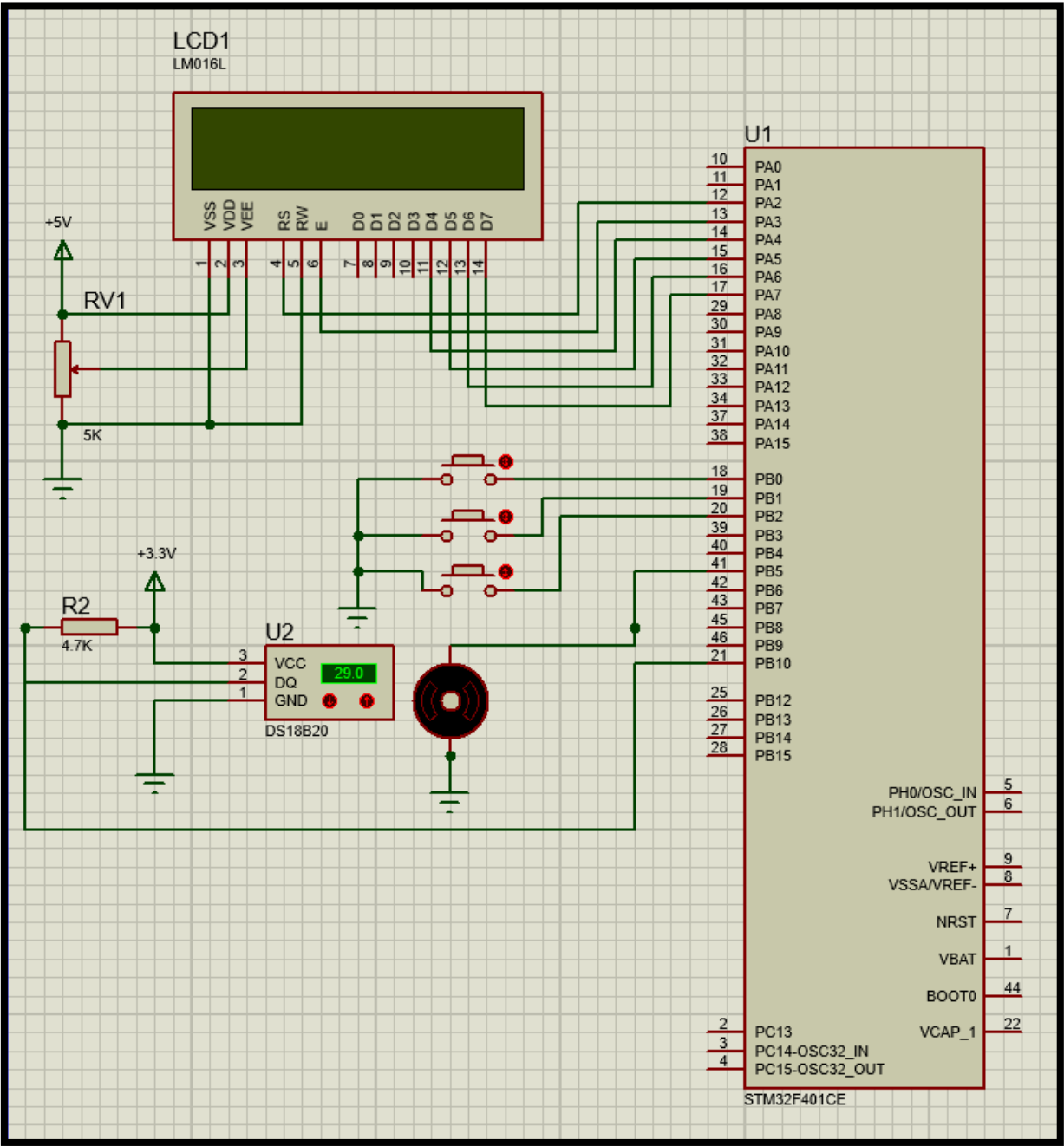


Figure D.1: Full Proteus schematic of the STM32-based thermostat controller

Appendix E. Additional Simulation Results

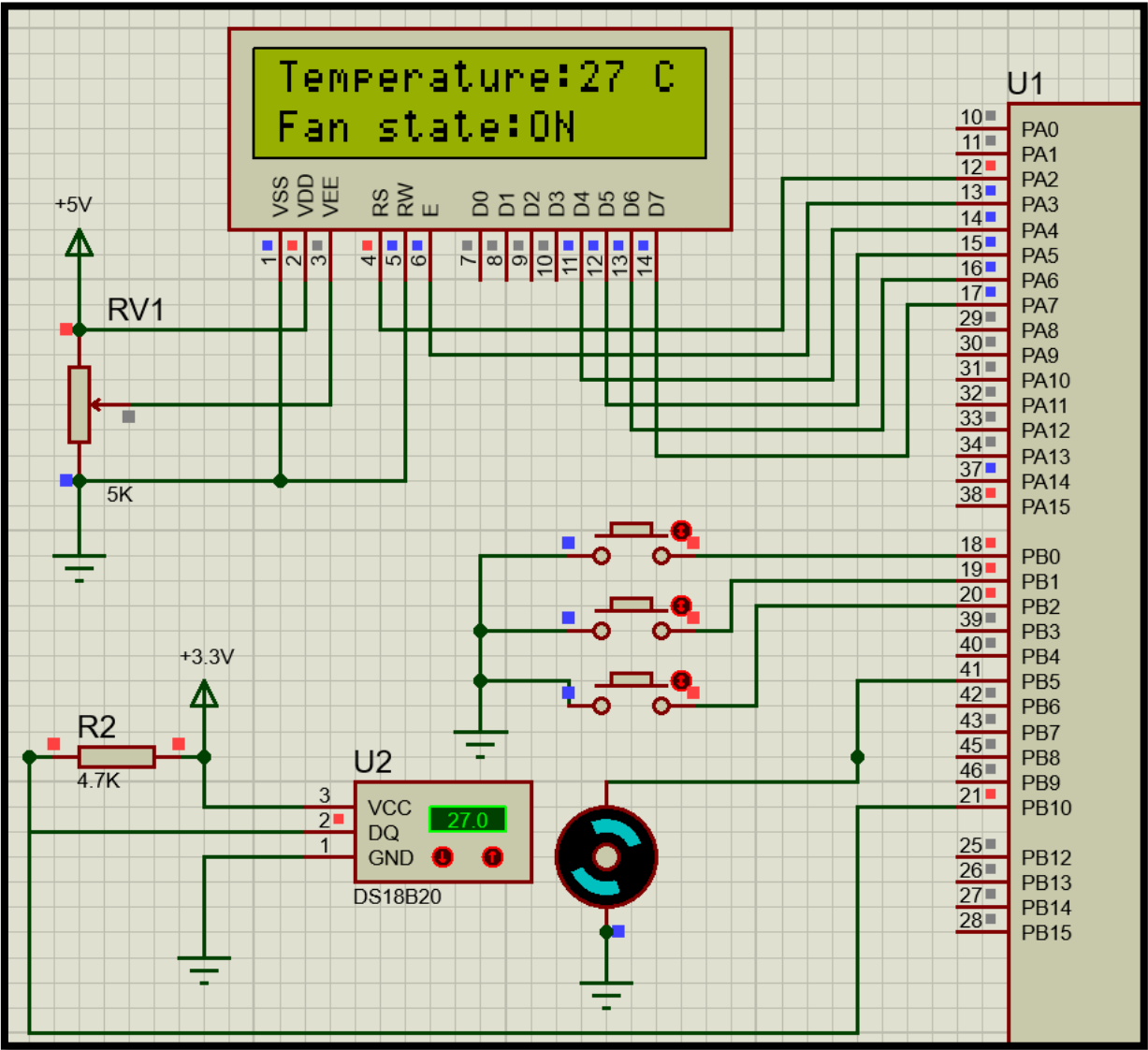


Figure E.1: Simulation result in normal mode with fan ON

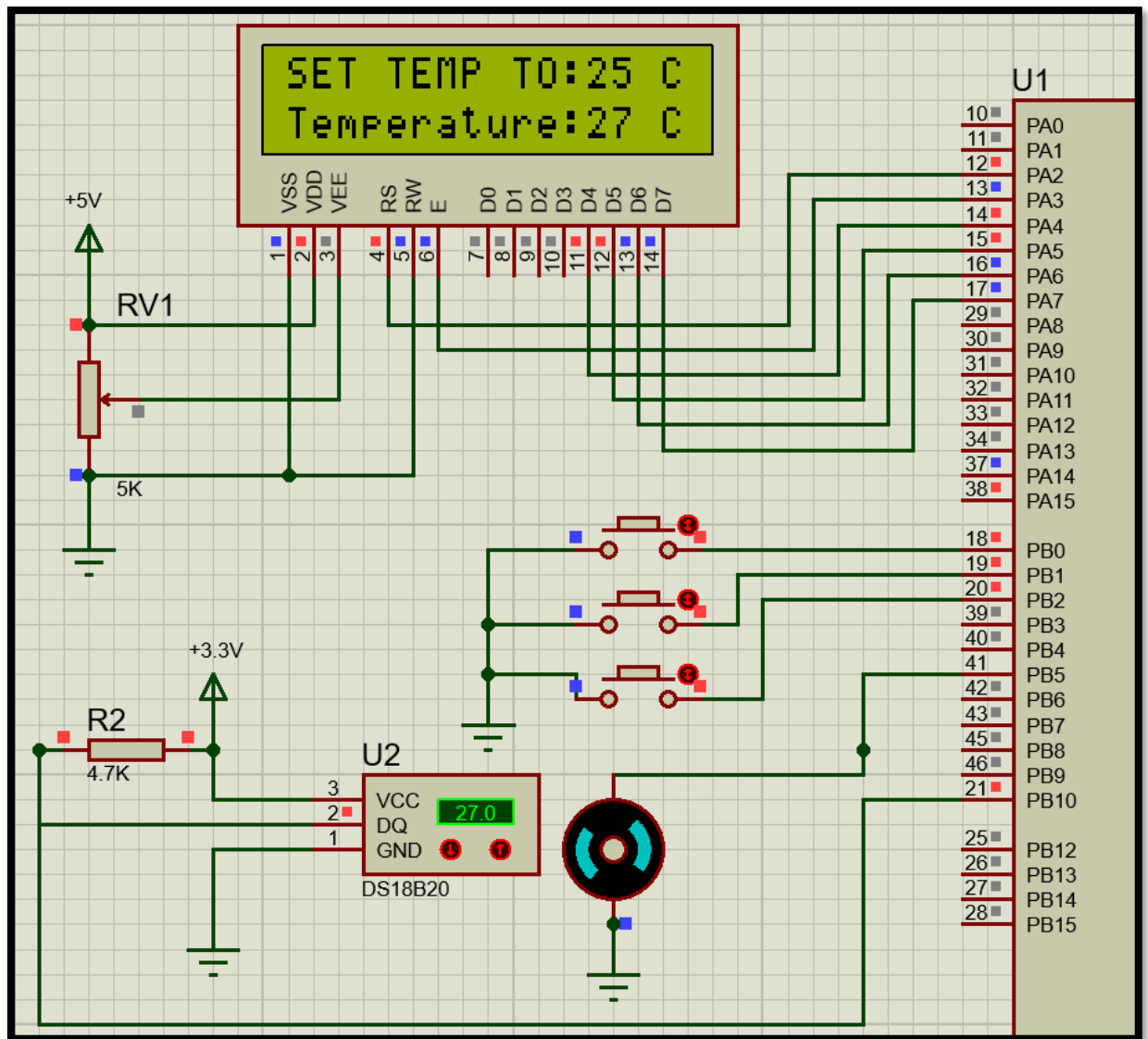


Figure E.2: Setpoint setting mode with 25°C target temperature

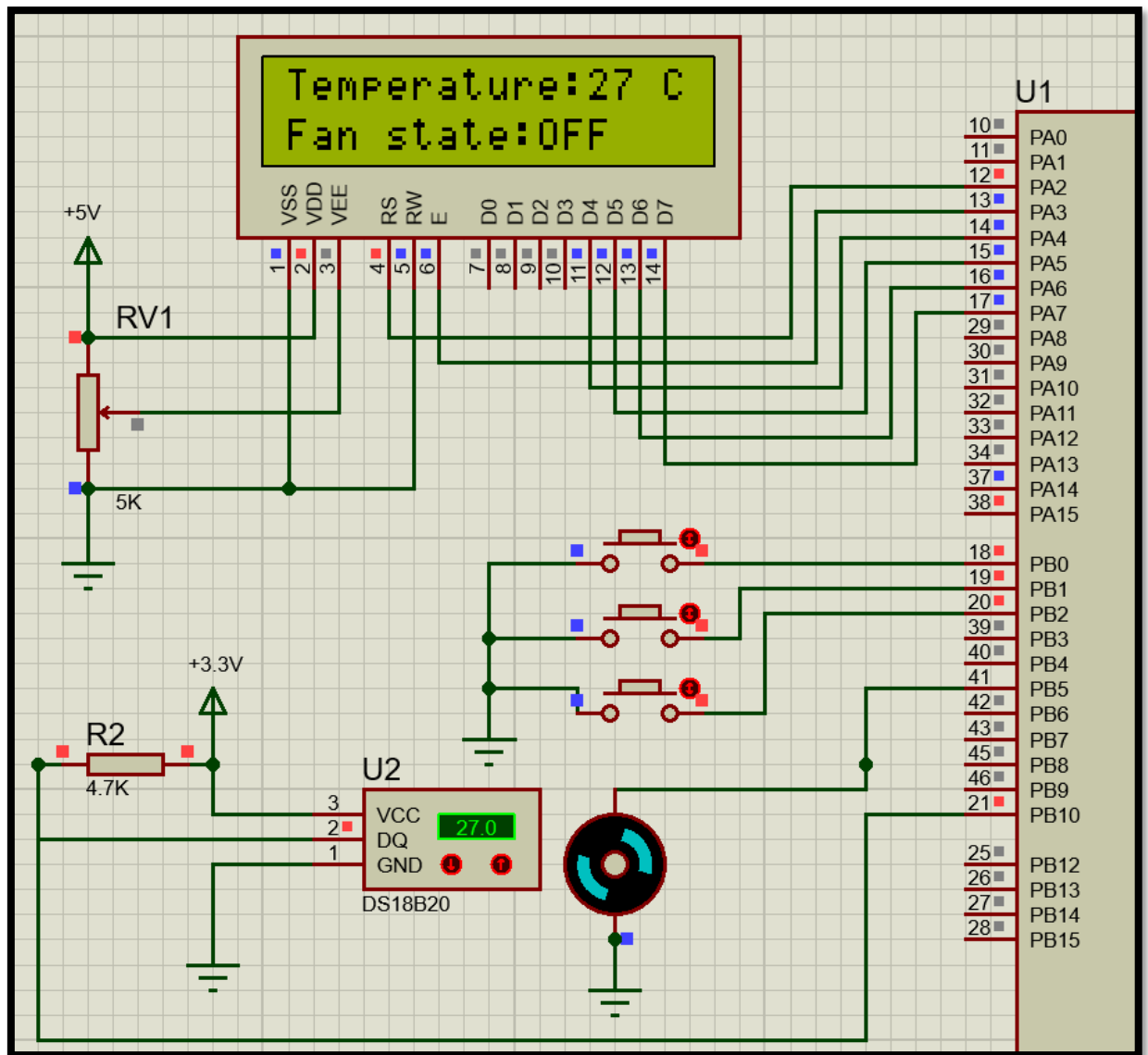


Figure E.3: Simulation result in normal mode with fan OFF

